

32. Ethernet Agent (ETHA)

32.1 Overview

Ethernet Agent (ETHA) consists of an agent interface module to allow communication within ESWM. ETHA handles data exchange between ESWM and an Ethernet PHY.

[Table 32.1](#) lists the ETHA specifications and [Figure 32.1](#) shows a ETHA block diagram.

Table 32.1 ETHA specifications

Item	Function	Description
Interfaces	Clock/Reset interface	Clock/Reset interface [TOP]
	Scan interface	Scan interface [TOP]
	Switch mode	Interface to receive the switch mode [MFWD]
	Pause interface	Interface to pause TX queues [COMA]
	SFR interface	Interface to access GWCA SFRs [COMA]
	Interrupts	GWCA interrupt to CPU
	Fabric interface	Interface to communicate with the data, TAG and pointer RAM [MFAB]
	Descriptor bus	Interface to receive frame information to send to CPU [MFWD]
	L2/L3 update bus	Interface to fetch the frame routing information [MFWD]
	BP request	Interface to receive BPs to store frames in the data, TAG and pointer RAM [COMA]
	BP release	Interface to release the BPs that has been used [COMA]
	TX timestamp capture interface	Interface to send TX timestamps to CPU [RMAC]
	RAM interfaces	Interface to communicate with RAMs [TOP]
Data provision	Ethernet frames	Corresponds to the IEEE 802.3, 802.1Q and 802.1CB standards [802.3] [802.1Q] [802.1CB]
	Descriptors	Local ethernet descriptor (ETHA to MFWD). Forwarding descriptor (MFWD to ETHA).

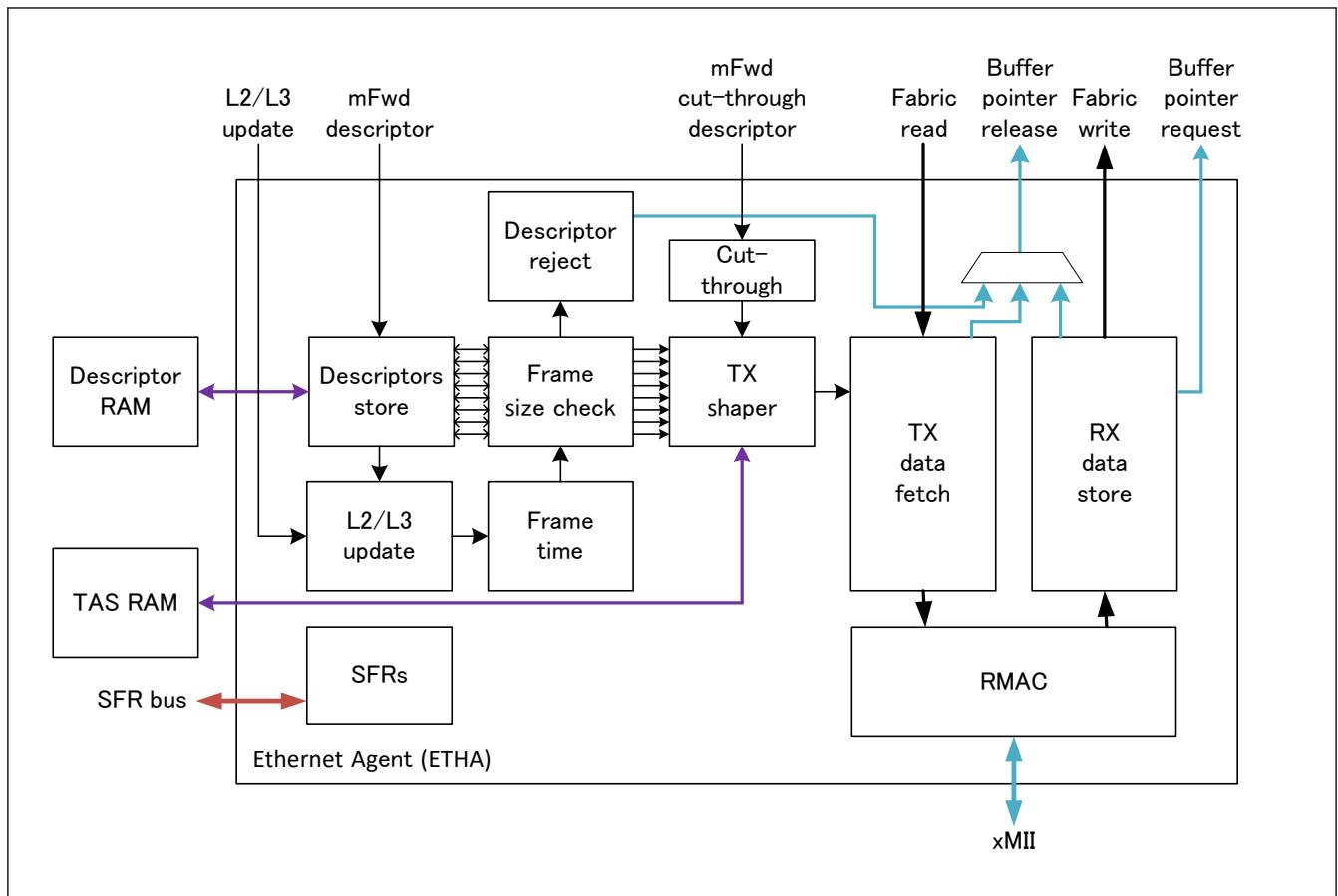


Figure 32.1 **ETHA block diagram**

Table 32.2 **ETHA functional blocks**

Block name	Function
RX data store	Transmits frames from RMAC to the local RAM
TX data fetch	Transmits frame from local RAM to RMAC
Cut-through	Receives cut-through descriptors from the forwarding engine
Descriptor store	Receives the descriptors from forwarding engine and controls the priority between descriptors
L2/L3 update	Fetches L2/L3 update information
Frame time	Calculates the frame time for TAS scheduling
Frame size check	Checks the frame size
Descriptor reject	Rejects descriptors
TX shaper	Shapes the traffic on TX side
RMAC	Mac interface [RMAC]
SFRs	ETHA SFRs

32.2 ETHA Register List

Table 32.3 **ETHA registers (1 of 3)**

Offset address	Register name	Abbreviation	Write access mode
0x0000	Mode Configuration Register	EAMC	Any
0x0004	Mode Status Register	EAMS	—
0x0010	IPV Remapping Configuration Register [802.1Q]	EAIRC	C
0x0014	TX Descriptor Queue Security Configuration Register	EATDQSC	C, O

Table 32.3 ETHA registers (2 of 3)

Offset address	Register name	Abbreviation	Write access mode
0x0018	TX Descriptor Queue Configuration Register	EATDQC	O
0x001C	TX Descriptor Queue Arbitration Configuration Register	EATDQAC	C
0x0020	TX Preemption Configuration Register	EATPEC	C
0x0040 + 0x04 × q	Transmission Maximum Frame Size Configuration Register q	EATMFSCq	C, O
0x0060 + 0x04 × q	Transmission Descriptor Queue Depth Configuration Register q	EATDQDCq	C
0x0080 + 0x04 × q	Transmission Descriptor Queue q Monitoring Register	EATDQMq	—
0x00A0 + 0x04 × q E: 0x00C0 + 0x04 × q	Transmission Descriptor Queue q Max Level Monitoring Register	EATDQMLMq	—
0x0100	Cut-Through Queue Configuration Register	EACTQC	C
0x0104	Cut-Through Descriptor Queue Depth Configuration Register	EACTDQDC	C
0x0108	Cut-Through Descriptor Queue Monitoring Register	EACTDQM	—
0x010C E: 0x0110	Cut-Through Descriptor Queue Max Level Monitoring Register	EACTDQMLM	—
0x0130	VLAN Control Configuration Register	EAVCC	C
0x0134	VLAN TAG Configuration Register	EAVTC	C
0x0138	Reception TAG Filtering Configuration Register	EARTFC	C
0x0200	CBS Admin Enable Configuration Register	EACAEC	C, O
0x0204	CBS Configuration Register	EACC	C, O
0x0220 + 0x04 × q	CBS Admin Increment Value Configuration Register q	EACAIVCq	C, O
0x0240 + 0x04 × q	CBS Admin Upper Limit Configuration Register q	EACAULCq	C, O
0x0260	CBS Oper Enable Monitoring Register	EACOEM	—
0x0280 + 0x04 × q	CBS Oper Increment Value Monitoring Register q	EACOIVMq	—
0x02A0 + 0x04 × q	CBS Oper Upper Limit Monitoring Register q	EACOULMq	—
0x02C0	CBS Gate State Monitoring Register	EACGSM	—
0x0300	TAS Configuration Register	EATASC	C, O
0x0304	TAS Initial Gate State Configuration Register	EATASIGSC	C, O
0x0320 + 0x04 × i	TAS Entry Number Configuration Register i	EATASENCi	C, O
0x0340	TAS Cut-Through Entry Number Configuration Register	EATASCCTENC	C, O
0x0360 + 0x04 × i	TAS Entry Number Monitoring Register i	EATASENMI	C, O
0x0380	Cut-Through Entry Number Monitoring Register	EATASCCTENM	C, O
0x03A0	TAS Cycle Start Time Configuration Register 0	EATASCSTC0	C, O
0x03A4	TAS Cycle Start Time Configuration Register 1	EATASCSTC1	C, O
0x03A8	TAS Cycle Start Time Monitoring Register 0	EATASCSTM0	C, O
0x03AC	TAS Cycle Start Time Monitoring Register 1	EATASCSTM1	C, O
0x03B0	TAS Cycle Time Configuration Register	EATASCTC	C, O
0x03B4	TAS Cycle Time Monitoring Register	EATASCTM	C, O
0x03C0	TAS Gate Learn Register 0	EATASGL0	C, O
0x03C4	TAS Gate Learn Register 1	EATASGL1	C, O
0x03C8	TAS Gate Learn Result Register	EATASGLR	—
0x03D0	TAS Gate Read Register	EATASGR	C, O
0x03D4	TAS Gate Read Result Register	EATASGRR	—
0x03E0	TAS Hardware Calibration Configuration Register	EATASHCC	C

Table 32.3 ETHA registers (3 of 3)

Offset address	Register name	Abbreviation	Write access mode
0x03E4	TAS RAM Initialization Register Monitoring Register	EATASRIRM	C, O
0x03E8	TAS Status Monitoring Register	EATASSM	—
0x0400 E: 0x0480	Under Minimum Frame Size Error Counter Register	EAUSMFSECN	—
0x0404 E: 0x0484	TAG Filtering Error Counter Register	EATFECN	—
0x0408 E: 0x0488	Frame Size Error Counter Register	EAFSECN	—
0x040C E: 0x048C	Descriptor Queue Overflow Error Counter Register	EADQOECN	—
0x0410 E: 0x0490	Descriptor Queue Security Error Counter Register	EADQSECN	—
0x0500	Error Interrupt Status Register 0	EAEIS0	C, O, D
0x0504	Error Interrupt Enable Register 0	EAEIE0	C, O, D
0x0508	Error Interrupt Disable Register 0	EAEID0	C, O, D
0x0510	Error Interrupt Status Register 1	EAEIS1	C, O, D
0x0514	Error Interrupt Enable Register 1	EAEIE1	C, O, D
0x0518	Error Interrupt Disable Register 1	EAEID1	C, O, D
0x0520	Error Interrupt Status Register 2	EAEIS2	C, O, D
0x0524	Error Interrupt Enable Register 2	EAEIE2	C, O, D
0x0528	Error Interrupt Disable Register 2	EAEID2	C, O, D

- Note:
- Base address: ETHA0 = 0x403C_A000, ETHA1 = 0x403C_C000, ETHA0_NS = 0x503C_A000, ETHA1_NS = 0x503C_C000
 - The address preceded by "E:" corresponds to an emulation address which allows a register to be read without modifying its content.
 - All registers can be read in any mode
 - Access modes:
 - Any: Register can be accessed in any mode
 - D: Register can be accessed in DISABLE mode
 - C: Register can be accessed in CONFIG mode
 - O: Register can be accessed in OPERATION mode.

32.3 ETHA Register Descriptions

32.3.1 ETHA Mode Function Registers

32.3.1.1 EAMC : Mode Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0000

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	OPC[1:0]	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Symbol	Function	R/W
1:0	OPC[1:0]	Operating Mode Command Used to set ETHA mode 0 0: Enter RESET mode 0 1: Enter DISABLE mode 1 0: Enter CONFIG mode 1 1: Enter OPERATION mode	R/W
31:2	—	These bits are read as 0.	R

Note: HW: This register is not writable if its value is different than EAMS.OPS.

32.3.1.2 EAMS : Mode Status Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0004

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	OPS[1:0]	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Symbol	Function	R/W
1:0	OPS[1:0]	Operating Mode Status Flag Indicates the current ETHA mode 0 0: RESET mode 0 1: DISABLE mode 1 0: CONFIG mode 1 1: OPERATION mode	R
31:2	—	These bits are read as 0.	R

32.3.2 Transmission Function Registers

32.3.2.1 EAIRC : IPV Remapping Configuration Register [802.1Q]

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0010

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	IPVR7[2:0]			—	IPVR6[2:0]			—	IPVR5[2:0]			—	IPVR4[2:0]		
Value after reset:	0	1	1	1	0	1	1	0	0	1	0	1	0	1	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	IPVR3[2:0]			—	IPVR2[2:0]			—	IPVR1[2:0]			—	IPVR0[2:0]		
Value after reset:	0	0	1	1	0	0	1	0	0	0	0	1	0	0	0	0

Bit	Symbol	Function	R/W
2:0	IPVR0[2:0]	IPV Remapping 0 Configure to which queue descriptor received with IPV 0 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 0).	R/W

Bit	Symbol	Function	R/W
3	—	This bit is read as 0.	R
6:4	IPVR1[2:0]	IPV Remapping 1 Configure to which queue descriptor received with IPV 1 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 1).	R/W
7	—	This bit is read as 0.	R
10:8	IPVR2[2:0]	IPV Remapping 2 Configure to which queue descriptor received with IPV 2 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 2).	R/W
11	—	This bit is read as 0.	R
14:12	IPVR3[2:0]	IPV Remapping 3 Configure to which queue descriptor received with IPV 3 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 3).	R/W
15	—	This bit is read as 0.	R
18:16	IPVR4[2:0]	IPV Remapping 4 Configure to which queue descriptor received with IPV 4 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 4).	R/W
19	—	This bit is read as 0.	R
22:20	IPVR5[2:0]	IPV Remapping 5 Configure to which queue descriptor received with IPV 5 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 5).	R/W
23	—	This bit is read as 0.	R
26:24	IPVR6[2:0]	IPV Remapping 6 Configure to which queue descriptor received with IPV 6 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 6).	R/W
27	—	This bit is read as 0.	R
30:28	IPVR7[2:0]	IPV Remapping 7 Configure to which queue descriptor received with IPV 7 is stored (when a descriptor is received from forwarding engine with FDESCR.IPV [MFWD] equal to 7).	R/W
31	—	This bit is read as 0.	R

32.3.2.2 EATDQSC : TX Descriptor Queue Security Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0014

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	TDQSL7 to TDQSL0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TDQSL7 to TDQSL0	TX Descriptor Queue n Security Level (n = 0 to 7) When a queue is secured, an unsecure descriptor cannot enter it (when a descriptor is from the forwarding engine with FDESCR.SEC [FWD] equal to 0). 0: Queue i unsecure 1: Queue i secure	R/W
31:8	—	These bits are read as 0.	R

32.3.2.3 EATDQC : TX Descriptor Queue Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0018

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	TDQP7 to TDQP0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TCTD QD	TDQD7 to TDQD0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TDQD7 to TDQD0	TX Descriptor Queue n Disable (n = 0 to 7) 0: Queue i enabled 1: Queue i disabled	R/W
8	TCTDQD	TX Cut-Through Descriptor Queue Disable 0: Cut-through queue enabled 1: Cut-through queue disabled	R/W ^{*1}
15:9	—	These bits are read as 0.	R
23:16	TDQP7 to TDQP0 ^{*2}	TX Descriptor Queue n Pause (n = 0 to 7) 0: Queue i active 1: Queue i paused	R/W ^{*1}
31:24	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

Note 2. SW: Pausing a queue for a long time can result in switch overflow. In case of a switch overflow [COMA], set this register to 0.

TDQDn bits (TX Descriptor Queue n Disable (n = 0 to 7))

Avoid descriptor from MFWD to enter the corresponding queue. The forwarding engine rejects the corresponding descriptor.

If disabled queue data is no longer required, the corresponding EATMFSCq.MFS register can be set to 0 to avoid more frames going to the RMAC.

[Setting condition]

- SW: Writing 1 to this bit.

[Clearing conditions]

- SW: Writing 0 to this bit
- HW: Exiting OPERATION mode clears this register (EAMC.OPC ≠ 11).

TCTDQD bit (TX Cut-Through Descriptor Queue Disable)

Avoid descriptor from MFWD to enter the cut-through queue by deasserting eha_ct_ready pin. The forwarding engine rejects the corresponding descriptor using the Common Agent reject bus [COMA].

[Setting condition]

- SW: Writing 1 to this bit.

[Clearing conditions]

- SW: Writing 0 to this bit
- HW: Exiting OPERATION mode clears this register (EAMC.OPC ≠ 11).

TDQPn bits (TX Descriptor Queue n Pause (n = 0 to 7))

Avoid frames to be sent to the RMAC [RMAC] by stopping descriptor fetching from the descriptor RAM.

[Setting condition]

- SW: Writing 1 to this bit.

[Clearing conditions]

- SW: Writing 0 to this bit
- HW: Exiting OPERATION mode clears this register (EAMC.OPC ≠ 11).

- Note:
- This register is used to stop a queue but MFWD does not stop sending descriptor to the corresponding queue. In this case the queue might overflow.
 - When a queue is paused, the CBS module is still active, so the credit value continues to increase.

32.3.2.4 EATDQAC : TX Descriptor Queue Arbitration Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x001C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	TDQA7[3:0]				TDQA6[3:0]				TDQA5[3:0]				TDQA4[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TDQA3[3:0]				TDQA2[3:0]				TDQA1[3:0]				TDQA0[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	TDQA0[3:0]	TX Descriptor Queue Arbitration 0 0: Queue 0 strict arbitration Others: Queue 0 WRR arbitration	R/W
7:4	TDQA1[3:0]	TX Descriptor Queue Arbitration 1 0: Queue 1 strict arbitration Others: Queue 1 WRR arbitration	R/W
11:8	TDQA2[3:0]	TX Descriptor Queue Arbitration 2 0: Queue 2 strict arbitration Others: Queue 2 WRR arbitration	R/W
15:12	TDQA3[3:0]	TX Descriptor Queue Arbitration 3 0: Queue 3 strict arbitration Others: Queue 3 WRR arbitration	R/W
19:16	TDQA4[3:0]	TX Descriptor Queue Arbitration 4 0: Queue 4 strict arbitration Others: Queue 4 WRR arbitration	R/W
23:20	TDQA5[3:0]	TX Descriptor Queue Arbitration 5 0: Queue 5 strict arbitration Others: Queue 5 WRR arbitration	R/W
27:24	TDQA6[3:0]	TX Descriptor Queue Arbitration 6 0: Queue 6 strict arbitration Others: Queue 6 WRR arbitration	R/W
31:28	TDQA7[3:0]	TX Descriptor Queue Arbitration 7 0: Queue 7 strict arbitration Others: Queue 7 WRR arbitration	R/W

- Note:
- For more details, see GWRDQAC register description [GWCA] in the GWCA specification
 - SW: In hybrid arbitration mode, all the queues with an EATDQAC.TDQAI value other than 0 should have a consecutive priority

- SW: In all arbitration mode except strict priority arbitration mode, at least two queues should have an EATDQAC.TDQAI value other than 0
- SW: WRR arbitration is only possible between queues with the same type (e or p), see EATPEC.TTQ register description (for example, all queues with an EATDQAC.TDQAI value other than 0 should have the same setting in EATPEC.TTQi register)
- SW: Only queues set to strict arbitration can use CBS functions
- SW: When using TAS, all queues not set to strict arbitration should be opened simultaneously.

32.3.2.5 EATPEC : TX Preemption Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0020

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	AFS[1:0]	
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	TTQ7 to TTQ0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TTQ7 to TTQ0	Transmission Type Queue n (n = 0 to 7) See section 32.5.1.7.2. Preemption . 0: Queue i contains e-frames 1: Queue i contains p-frames	R/W
15:8	—	These bits are read as 0.	R
17:16	AFS[1:0]	Additional Fragment Size Used for preemption. It defines the minimum size of a fragment which is not the last fragment of a frame. 0 0: 0 byte is added to minimum fragment size (minimum fragment size = 64 bytes) 0 1: 64 bytes are added to minimum fragment size (minimum fragment size = 128 bytes) 1 0: 128 bytes are added to minimum fragment size (minimum fragment size = 192 bytes) 1 1: 192 bytes are added to minimum fragment size (minimum fragment size = 256 bytes)	R/W
31:18	—	These bits are read as 0.	R

32.3.2.6 EATMFSCq : Transmission Maximum Frame Size Configuration Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0040 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	MFS[15:0]															
Value after reset:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
15:0	MFS[15:0]	Maximum Frame Size Maximum frame size for descriptor queue q. All bigger frames are rejected.	R/W
31:16	—	These bits are read as 0.	R

Note: HW: The MFS size comparison takes the frame size on the PHY interface.

32.3.2.7 EATDQDCq : Transmission Descriptor Queue Depth Configuration Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0060 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DQD[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DQD[9:0]	Descriptor Queue Depth Set the number of descriptors that can contain descriptor queue q. For details, see GWRDQDCq register description [GWCA] in the GWCA specification.	R/W
31:10	—	These bits are read as 0.	R

Note: SW: The sum of DQD fields should always be smaller or equal to 512.

32.3.2.8 EATDQMq : Transmission Descriptor Queue q Monitoring Register (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0080 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DNQ[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DNQ[9:0]	Descriptor Number in Queue These bits indicate the current number of descriptors stored in TX descriptor queue q. All descriptors stored in the TX descriptor queue q are rejected/dropped when exiting OPERATION mode.	R
31:10	—	These bits are read as 0.	R

DNQ[9:0] bits (Descriptor Number in Queue)

[Increment condition]

- HW: Incremented by 1 when a descriptor is received from the MFWD for the corresponding queue, the queue is not full and there is no descriptor security error.

[Decrement condition]

- HW: Decrement by 1 when a descriptor is read by Ethernet Agent to send data to RMAC [RMAC] or to reject data.

32.3.2.9 EATDQMLMq : Transmission Descriptor Queue q Max Level Monitoring Register (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x00A0 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	DMLQ[9:0]									
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
9:0	DMLQ[9:0]	Descriptor Max Level in Queue These bits indicate the maximum number of descriptors that is stored in TX descriptor queue q.	R
31:10	—	These bits are read as 0.	R

DMLQ[9:0] bits (Descriptor Max Level in Queue)

[Clearing conditions]

- HW: RESET mode clears this register (EAMC.OPS == 00)
- SW: Reading this register clears to EATDQMq.DNQ.

[Increment condition]

- HW: Increments to EATDQMq.DNQ value when smaller than EATDQMq.DNQ.

32.3.2.10 EACTQC : Cut-Through Queue Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0100

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CTQD[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	CTQD[15:0] ^{*1}	Cut-Through Queue Delay (clk Cycle Number) Set the number of clocks to wait for descriptor transmission.	R/W

Bit	Symbol	Function	R/W
31:16	—	These bits are read as 0.	R

Note 1. Set to 0 for this product.

32.3.2.11 EACTDQDC : Cut-Through Descriptor Queue Depth Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0104

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CTDQD[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	CTDQD[3:0]	Cut-Through Descriptor Queue Depth Number of descriptors that can contain cut-through descriptor queue.	R/W
31:4	—	These bits are read as 0.	R

Note: SW: The maximum value of this register is 8.

32.3.2.12 EACTDQM : Cut-Through Descriptor Queue Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0108

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CTQDN[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	CTQDN[3:0]	Cut-Through Queue Descriptor Number	R
31:4	—	These bits are read as 0.	R

CTQDN[3:0] bits (Cut-Through Queue Descriptor Number)

These bits indicate the current number of descriptors stored in the cut-through descriptor queue.

All cut-through descriptors stored in the queue are not rejected/dropped when exiting OPERATION mode. But the frame being received by ETHA is rejected/dropped, if ETHA transitioned to reset before the frame is completely received.

All cut-through descriptors stored in the queue are not rejected/dropped when exiting OPERATION mode but are rejected/dropped when transitioning to RESET mode.

[Increment condition]

- HW: Incremented by 1 when a descriptor is received from the forwarding engine for the cut-through queue and the queue is not full.

[Decrement condition]

- HW: Decrement by 1 when a descriptor is read by Ethernet Agent to send data to RMAC [RMAC].

32.3.2.13 EACTDQMLM : Cut-Through Descriptor Queue Max Level Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x010C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CTDMLQ[3:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
3:0	CTDMLQ[3:0]	Cut-Through Descriptor Max Level in Queue These bits indicate the maximum number of descriptors that is stored in cut-through descriptor queue.	R
31:4	—	These bits are read as 0.	R

CTDMLQ[3:0] bits (Cut-Through Descriptor Max Level in Queue)

[Clearing conditions]

- HW: RESET mode clears this register (EAMC.OPS == 00)
- SW: Reading this register clears to EACTDQM.CTQDN.

[Increment condition]

- HW: Increments to EACTDQM.CTQDN value when smaller than EACTDQM.CTQDN.

32.3.3 TAG Function Registers

32.3.3.1 EAVCC : VLAN Control Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0130

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	VEM[2:0]		
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	VIM
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	VIM	VLAN Ingress Mode 0: Incoming VLAN mode 1: Port based VLAN mode	R/W
15:1	—	These bits are read as 0.	R
18:16	VEM[2:0]	VLAN Egress Mode 0 0 0: No VLAN mode 0 0 1: C-TAG VLAN mode 0 1 0: HW C-TAG VLAN mode 0 1 1: SC-TAG VLAN mode 1 0 0: HW SC-TAG VLAN mode	R/W
31:19	—	These bits are read as 0.	R

32.3.3.2 EAVTC : VLAN TAG Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0134

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	STD	STP[2:0]			STV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CTD	CTP[2:0]			CTV[11:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
11:0	CTV[11:0]	C-TAG VLAN Set the C-TAG VID values.	R/W
14:12	CTP[2:0]	C-TAG PCP Set the C-TAG PCP values.	R/W
15	CTD	C-TAG DEI Set the C-TAG DEI values.	R/W
27:16	STV[11:0]	S-TAG VLAN Set the S-TAG VID values.	R/W
30:28	STP[2:0]	S-TAG PCP Set the S-TAG PCP values.	R/W
31	STD	S-TAG DEI Set the S-TAG DEI values.	R/W

32.3.3.3 EARTFC : Reception TAG Filtering Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0138

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	UT	SCRT	SCT	CRT	CT	CSRT	CST	RT	NT
Value after reset:	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	NT	No TAG 0: No TAG frame passed 1: No TAG frame rejected	R/W
1	RT	R-TAG 0: R-TAG frame passed 1: R-TAG frame rejected	R/W
2	CST	CoS-TAG 0: CoS-TAG frame passed 1: CoS-TAG frame rejected	R/W
3	CSRT	CoSR-TAG 0: CoSR-TAG frame passed 1: CoSR-TAG frame rejected	R/W
4	CT	C-TAG 0: C-TAG frame passed 1: C-TAG frame rejected	R/W
5	CRT	CR-TAG 0: CR-TAG frame passed 1: CR-TAG frame rejected	R/W
6	SCT	SC-TAG 0: SC-TAG frame passed 1: SC-TAG frame rejected	R/W
7	SCRT	SCR-TAG 0: SCR-TAG frame passed 1: SCR-TAG frame rejected	R/W
8	UT	Unknown TAG 0: Unknown TAG frame passed 1: Unknown TAG frame rejected	R/W
31:9	—	These bits are read as 0.	R

32.3.4 CBS Function Registers [802.1Qav]

32.3.4.1 EACAEC : CBS Admin Enable Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0200

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CE7 to CE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CE7 to CE0	CBS Enable n (n = 0 to 7) This register is only used to configure CBS and is not directly used by CBS modules. 0: CBS for descriptor queue i disabled 1: CBS for descriptor queue i enabled	R/W
31:8	—	These bits are read as 0.	R

32.3.4.2 EACC : CBS Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0204

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CC7 to CC0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CC7 to CC0	Configuration Change n (n = 0 to 7) 0: No effect 1: The Admin settings of CBS i are copied to its Oper settings (EACAEC.CEi, EACAIVCq.CIV, and EACAULCq.CUL are respectively copied to EACOEM.CEi, EACOIVMq.CIV, and EACOULMq.CUL)	W*1
31:8	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

32.3.4.3 EACAIVCq : CBS Admin Increment Value Configuration Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0220 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CIV[19:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CIV[19:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
19:0	CIV[19:0]	Credit Increment Value	R/W
31:20	—	These bits are read as 0.	R

CIV bits (Credit Increment Value)

Configure CBS throughput.

CIV[3:0] is the credit increment in byte per clock and CIV[15:0] is the credit increment in sub-byte per clock.

This register is only used to configure CBS and is not directly used by CBS modules.

For more details, see [section 32.5.1.5. CBS \(Credit-based shaper\) \[802.1Qav\]](#).

Caution: **SW: Setting this register to 0 or a value close to 0 sets the corresponding transmission queue to a very low throughput and can lead to a queue overflow.**

32.3.4.4 EACAULCq : CBS Admin Upper Limit Configuration Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0240 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	CUL[30:0]														
Value after reset:	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CUL[30:0]															
Value after reset:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
30:0	CUL[30:0]	Credit Upper Limit	R/W
31	—	This bit is read as 0.	R

CUL bits (Credit Upper Limit)

Configure queue q interference time.

This register is only used to configure CBS and is not directly used by CBS modules.

For more details, see [section 32.5.1.5. CBS \(Credit-based shaper\) \[802.1Qav\]](#).

32.3.4.5 EACOEM : CBS Oper Enable Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0260

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CE7 to CE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CE7 to CE0	CBS Enable n (n = 0 to 7) [Update conditions] HW: This register is updated to EACACC.CEi when 1 is written to EACC.CCi. 0: CBS for descriptor queue i disabled 1: CBS for descriptor queue i enabled	R
31:8	—	These bits are read as 0.	R

32.3.4.6 EACOIVMq : CBS Oper Increment Value Monitoring Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0280 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CIV[19:0]			
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CIV[19:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
19:0	CIV[19:0]	Credit Increment Value	R
31:20	—	These bits are read as 0.	R

CIV bits (Credit Increment Value)

Configure CBS throughput.

CIV[3:0] is the credit increment in byte per clock and CIV[15:0] is the credit increment in sub-byte per clock.

This register is only used to configure CBS and is not directly used by CBS modules.

[Update condition]

- HW: This register is updated to EACAIVq.CIV when 1 is written to EACC.CCi.

32.3.4.7 EACOULMq : CBS Oper Upper Limit Monitoring Register q (q = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x02A0 + 0x4 × q

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	CUL[30:0]														
Value after reset:	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	CUL[30:0]															
Value after reset:	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
30:0	CUL[30:0]	Credit Upper Limit	R
31	—	This bit is read as 0.	R

CUL bits (Credit Upper Limit)

Configure queue q interference time.

This register is only used to configure CBS and is not directly used by CBS modules.

[Update condition]

- HW: This register is updated to EACAULq.CUL when 1 is written to EACC.CCi.

32.3.4.8 EACGSM : CBS Gate State Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x02C0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CGS7 to CGS0							
Value after reset:	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
7:0	CGS7 to CGS0	CBS Gate State n (n = 0 to 7) 0: CBS does not authorize queue i transmission (CBS credit negative) 1: CBS authorizes queue i transmission (CBS credit positive)	R
31:8	—	These bits are read as 0.	R

32.3.5 TAS Function Registers [802.1Qbv]

32.3.5.1 EATASC : TAS Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0300

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	TASCA[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASTS	—	—	—	—	—	TASCI	TASC C	TASE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	TASE	TAS Enable Enables the TAS schedule See section 32.5.1.6.1. TAS General Behavior . 0: TAS disabled 1: TAS enabled	R/W ^{*1}
1	TASCC	TAS Configuration Change 0: TAS is not changing configuration 1: TAS is changing configuration	R/W ^{*1}
2	TASCI	TAS Configuration Impossible This bit aids the software in detecting when TAS configuration is possible, it is equal to (EATASC.TASE & ~ EATASSM.TASSO) EATASC.TASCC. 0: TAS configuration is possible 1: TAS configuration is not possible	R
7:3	—	These bits are read as 0.	R
8	TASTS	TAS Timer Select Selects the gPTP to be used for TAS module.	R/W
15:9	—	These bits are read as 0.	R
23:16	TASCA[7:0]	TAS Configuration Address Shows the address from which entry learning occurs in TAS RAM for the next configuration.	R
31:24	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

TASE bit (TAS Enable)

[Setting condition]

- SW: Writing 1 to this bit.

[Clearing conditions]

- SW: Writing 0 to this bit
- HW: When in RESET mode.

TASCC bit (TAS Configuration Change)

[Setting condition]

- SW: Writing 1 to this bit.

[Clearing conditions]

- HW: When TAS configuration change is complete

- HW: When in RESET mode.

See [section 32.5.1.6.1. TAS General Behavior](#).

TASCI bit (TAS Configuration Impossible)

[Clearing condition]

- HW: When in RESET mode.

TASCA[7:0] bits (TAS Configuration Address)

[Clearing condition]

- HW: When in RESET mode.

32.3.5.2 EATASIGSC : TAS Initial Gate State Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0304

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASC TIGS	TASIGS7 to TASIGS0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TASIGS7 to TASIGS0	TAS Initial Gate State n (n = 0 to 7) Bit q is the gate state used for initialization of descriptor queue q gate. 0: Initial gate state is closed 1: Initial gate state is opened	R/W
8	TASC TIGS	TAS Cut-Through Initial Gate State Gate state used for initialization of cut-through gate. 0: Initial gate state is closed 1: Initial gate state is opened	R/W
31:9	—	These bits are read as 0.	R

32.3.5.3 EATASENCi : TAS Entry Number Configuration Register i (i = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0320 + 0x4 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASAE[8:0]								
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
8:0	TASAEN[8:0]	TAS Admin Entry Number This register is only used to configure TAS and is not directly used by TAS module. Sets the number of TAS RAM entries used for gate i schedule.	R/W
31:9	—	These bits are read as 0.	R

- Note:
- SW: If the TAS module is dynamically used (TAS setting flow in [Figure 32.11](#) is used when EATASC.TASE is already set), the sum of EATASENCi.TASAEN should be smaller or equal to 119.
 - SW: If the TAS module is statistically used (TAS setting flow in [Figure 32.11](#) is not used when EATASC.TASE is already set), the sum of EATASENCi.TASAEN should be smaller or equal to 247.

32.3.5.4 EATASCTENC : TAS Cut-Through Entry Number Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0340

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASCTAEN[8:0]								
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
8:0	TASCTAEN[8:0]	TAS Admin Cut-Through Entry Number This register is only used to configure TAS and is not directly used by TAS module. Sets the number of TAS RAM entries used for cut-through gate schedule.	R/W
31:9	—	These bits are read as 0.	R

- Note:
- SW: If the TAS module is dynamically used (TAS setting flow in [Figure 32.11](#) is used when EATASC.TASE is already set), the sum of EATASENCi.TASAEN and EATASCTENC.TASCTAEN should be smaller or equal to 119.
 - SW: If the TAS module is statistically used (TAS setting flow in [Figure 32.11](#) is not used when EATASC.TASE is already set), the sum of EATASENCi.TASAEN and EATASCTENC.TASCTAEN should be smaller or equal to 247.

32.3.5.5 EATASENMi : TAS Entry Number Monitoring Register i (i = 0 to 7)

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0360 + 0x4 × i

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASOEN[8:0]								
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
8:0	TASOEN[8:0]	TAS Oper Entry Number	R
31:9	—	These bits are read as 0.	R

TASOEN[8:0] bits (TAS Oper Entry Number)

[Update condition]

- HW: This register is updated to EATASENCi.TASAEN when configuration change occurs or when schedule start occurs. See [section 32.5.1.6.1. TAS General Behavior](#).

[Clearing condition]

- HW: When in RESET mode.

32.3.5.6 EATASCTENM : TAS Cut-Through Entry Number Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0380

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASCTOEN[8:0]								
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
8:0	TASCTOEN[8:0]	TAS Cut-Through Oper Entry Number	R
31:9	—	These bits are read as 0.	R

TASCTOEN[8:0] bits (TAS Cut-Through Oper Entry Number)

[Update condition]

- HW: This register is updated to EATASCTENC.TASCTAEN when configuration change occurs or when schedule start occurs. See [section 8.1.6.1](#).

[Clearing condition]

- HW: When in RESET mode.

32.3.5.7 EATASCSTC0 : TAS Cycle Start Time Configuration Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03A0

Bit position:	31	0																															
Bit field:	TASACSTP0[31:0]																																
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	TASACSTP0[31:0]	TAS Admin Cycle Start Time Part 0 Time at which TAS scheduler should start/change configuration. This register is in ns.	R/W

32.3.5.8 EATASCSTC1 : TAS Cycle Start Time Configuration Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03A4

Bit position: 31 0

Bit field: TASACSTP1[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TASACSTP1[31:0]	TAS Admin Cycle Start Time Part 1 Time at which TAS scheduler should start/change configuration. This register is in $2^{32} \times \text{ns}$.	R/W

32.3.5.9 EATASCSTM0 : TAS Cycle Start Time Monitoring Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03A8

Bit position: 31 0

Bit field: TASOCSTP0[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TASOCSTP0[31:0]	TAS Oper Cycle Start Time Part 0	R

TASOCSTP0[31:0] bits (TAS Oper Cycle Start Time Part 0)

[Update conditions]

- HW: {EATASCSTM1, EATASCSTM0} is updated to {EATASCSTC1, EATASCSTC0} + EATASCTC when configuration change occurs or when schedule start occurs. See [section 32.5.1.6.1. TAS General Behavior](#).
- HW: This register is updated to the next cycle start time every time a new cycle starts.

[Clearing condition]

- HW: When in RESET mode.

32.3.5.10 EATASCSTM1 : TAS Cycle Start Time Monitoring Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03AC

Bit position: 31 0

Bit field: TASOCSTP1[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TASOCSTP1[31:0]	TAS Oper Cycle Start Time Part 1	R

TASOCSTP1[31:0] bits (TAS Oper Cycle Start Time Part 1)

[Update conditions]

- HW: {EATASCSTM1, EATASCSTM0} is updated to the value of {EATASCSTC1, EATASCSTC0} + EATASCTC when configuration change occurs or when schedule start occurs. See [section 32.5.1.6.1. TAS General Behavior](#).
- HW: This register is updated to the next cycle start time every time a new cycle starts.

[Clearing condition]

- HW: When in RESET mode.

32.3.5.11 EATASCTC : TAS Cycle Time Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03B0

Bit position: 31

0

Bit field:

TASACT[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TASACT[31:0]	TAS Admin Cycle Time Configure the cycle time for TAS in ns.	R/W

Note: SW: Set this register to a value greater than 100 ns.

32.3.5.12 EATASCTM : TAS Cycle Time Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03B4

Bit position: 31

0

Bit field:

TASOCT[31:0]

Value after reset: 0

Bit	Symbol	Function	R/W
31:0	TASOCT[31:0]	TAS Oper Cycle Time	R

TASOCT[31:0] bits (TAS Oper Cycle Time)

[Update condition]

- HW: This register is updated to EATASCTC.TASACT when configuration change occurs or when schedule start occurs. See [section 32.5.1.6.1. TAS General Behavior](#).

[Clearing condition]

- HW: When in RESET mode.

32.3.5.13 EATASGL0 : TAS Gate Learn Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03C0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	TASGAL[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TASGAL[7:0]	TAS Gate Address Learn Configures the address in which the TAS entry is learned.	R/W
31:8	—	These bits are read as 0.	R

32.3.5.14 EATASGL1 : TAS Gate Learn Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03C4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	TASGSL	TASGTL[27:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TASGTL[27:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
27:0	TASGTL[27:0]	TAS Gate Time Learn Configures gate time	R/W
28	TASGSL	TAS Gate State Learn Configures gate state	R/W
31:29	—	These bits are read as 0.	R

32.3.5.15 EATASGLR : TAS Gate Learn Result Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03C8

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	GL	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
30:0	—	These bits are read as 0.	R
31	GL	Gate Learn	R

GL bit (Gate Learn)

[Setting condition]

- HW: Writing to the EATASGL1 register.

[Clearing condition]

- HW: When learning is completed, this bit is deasserted.

32.3.5.16 EATASGR : TAS Gate Read Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03D0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	TASGAR[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	TASGAR[7:0]	TAS Gate Address Read Configures the address in which the TAS entry is read.	R/W
31:8	—	These bits are read as 0.	R

32.3.5.17 EATASGRR : TAS Gate Read Result Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03D4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	GR	—	—	TASGSR	TASGTR[27:0]											
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TASGTR[27:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
27:0	TASGTR[27:0]	TAS Gate Time Read Displays gate time read value.	R
28	TASGSR	TAS Gate State Read Displays gate state read value.	R
30:29	—	These bits are read as 0.	R
31	GR	Gate Read	R

TASGTR bits (TAS Gate Time Read)

[Update condition]

- EATASGRR.GR clear event

TASGSR bit (TAS Gate State Read)

[Update condition]

- EATASGRR.GR clear event

GR bit (Gate Read)

[Setting condition]

- HW: Writing to the EATASGR register.

[Clearing condition]

- HW: When reading is completed, this bit is deasserted.

32.3.5.18 EATASHCC : TAS Hardware Calibration Configuration Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03E0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TASJ[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TASJ[15:0]	TAS Jitter Configure the jitter between the TAS data transmission and RMAC PHY interface [RMAC]. See section 32.5.1.6.3. TAS Hardware Calibration .	R/W
31:16	—	These bits are read as 0.	R

32.3.5.19 EATASRIRM : TAS RAM Initialization Register Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03E4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	TASR R	TASRI OG
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	TASRIOG	TAS RAM Initialization Ongoing	R/W ¹
1	TASRR	TAS RAM Ready	R
31:2	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

TASRIOG bit (TAS RAM Initialization Ongoing)

[Setting condition]

- SW: Writing 1 to this register starts TAS RAM initialization.

[Clearing condition]

- HW: When TAS RAM initialization is completed.

TASRR bit (TAS RAM Ready)

[Setting condition]

- When EATASRIRM.TASRIOG is cleared.

[Clearing condition]

- Writing 1 to EATASRIRM.TASRIOG.

32.3.5.20 EATASSM : TAS Status Monitoring Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x03E8

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	TASS O
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	TASC TGS	TASGS7 to TASGS0							
Value after reset:	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1

Bit	Symbol	Function	R/W
7:0	TASGS7 to TASGS0	TAS Gate n State (n = 0 to 7) 0: Gate i is closed 1: Gate i is opened	R
8	TASCTGS*1	TAS Cut-Through Gate State 0: Cut-through gate is closed 1: Cut-through gate is opened	R
15:9	—	These bits are read as 0.	R
16	TASSO	TAS Scheduler Ongoing 0: TAS scheduler is not ongoing 1: TAS scheduler is ongoing	R
31:17	—	These bits are read as 0.	R

Note 1. HW: This register only exists if UCIAPRACE020_CT_ON is defined.

TASSO bit (TAS Scheduler Ongoing)

[Clearing condition]

- HW: When in RESET mode.

32.3.6 ETHA Counter Registers

32.3.6.1 EAUSMFSECN : Under Minimum Frame Size Error Counter Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0400

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	USMFSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	USMFSEN[15:0]	Under Minimum Frame Size Error Number This register counts the number of received data lost because of under minimum size error.	R

Bit	Symbol	Function	R/W
31:16	—	These bits are read as 0.	R

USMFSEN[15:0] bits (Under Minimum Frame Size Error Number)

[Clearing conditions]

- HW: When in RESET mode
- SW: Reading this register.

[Increment condition]

- HW: Incremented by 1 when an Under Minimum Frame Size Error occurs and if this register has a value other than 0xFFFF.

32.3.6.2 EATFECN : TAG Filtering Error Counter Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0404

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TFEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	TFEN[15:0]	TAG Filtering Error Number This register counts the number of received data lost because of TAG filtering.	R
31:16	—	These bits are read as 0.	R

TFEN[15:0] bits (TAG Filtering Error Number)

[Clearing conditions]

- HW: When in RESET mode
- SW: Reading this register.

[Increment condition]

- HW: Incremented by 1 when a TAG filter error occurs and if this register has a value other than 0xFFFF.

32.3.6.3 EAFSECN : Frame Size Error Counter Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0408

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	FSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	FSEN[15:0]	Frame Size Error Number This register counts the number of received data lost because of a Frame Size Error.	R
31:16	—	These bits are read as 0.	R

FSEN[15:0] bits (Frame Size Error Number)

[Clearing conditions]

- HW: When in RESET mode
- SW: Reading this register.

[Increment condition]

- HW: Incremented by 1 when a Frame Size Error occurs and if this register has a value other than 0xFFFF.

32.3.6.4 EADQOECN : Descriptor Queue Overflow Error Counter Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x040C

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DQOEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DQOEN[15:0]	Descriptor Queue Overflow Error Number This register counts the number of transmit data lost because of a Descriptor Queue Overflow Error.	R
31:16	—	These bits are read as 0.	R

DQOEN[15:0] bits (Descriptor Queue Overflow Error Number)

[Clearing conditions]

- HW: When in RESET mode
- SW: Reading this register.

[Increment condition]

- HW: Incremented by 1 when a Descriptor Queue Overflow Error occurs and if this register has a value other than 0xFFFF.

32.3.6.5 EADQSECN : Descriptor Queue Security Error Counter Register

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0410

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	DQSEN[15:0]															
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
15:0	DQSEN[15:0]	Descriptor Queue Security Error Number This register counts the number of transmit data lost because of a Descriptor Queue Security Error.	R
31:16	—	These bits are read as 0.	R

DQSEN[15:0] bits (Descriptor Queue Security Error Number)

[Clearing conditions]

- HW: When in RESET mode
- SW: Reading this register.

[Increment condition]

- HW: Incremented by 1 when a Descriptor Queue Security Error occurs and if this register has a value other than 0xFFFF.

32.3.7 ETHA Interrupt Registers

32.3.7.1 Error Interrupt Registers

32.3.7.1.1 EAEIS0 : Error Interrupt Status Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0500

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	FSES7 to FSES0								—	TFES	USMF SES	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	—	These bits are read as 0.	R
5	USMFSES	Under Switch Minimum Frame Size Error Status Flag	R/W ^{*1}
6	TFES	TAG Filtering Error Status Flag	R/W ^{*1}
7	—	This bit is read as 0.	R
15:8	FSES7 to FSES0	Frame Size Error Status Flag n (n = 0 to 7)	R/W ^{*1}
31:16	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

USMFSES bit (Under Switch Minimum Frame Size Error Status Flag)

[Setting condition]

- HW: A frame smaller than 32 bytes has been received from RMAC.

This error occurs with [RMAC] MEIS.RPOES.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: The frame is discarded
- SW: System-dependent, therefore cannot be defined.

TFES bit (TAG Filtering Error Status Flag)

[Setting condition]

- HW: An unauthorized TAG format has been detected. (See [section 32.5.2.1. RX Data Store](#)).

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: Frame is sent to MFWD with local descriptor DESCR.TFE bit set. (See [section 32.5.2.1. RX Data Store](#)).
- SW: System-dependent, therefore cannot be defined.

FSESn bits (Frame Size Error Status Flag n (n = 0 to 7))

[Setting condition]

- HW: Bit q of this register is set when a frame bigger than EATMFSC.MFS has been received for descriptor queue q.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: The current frame is lost and the following data is processed normally
- SW: System-dependent, therefore it cannot be defined.

32.3.7.1.2 EAEIE0 : Error Interrupt Enable Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0504

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	FSEE7 to FSEE0								—	TFEE	USMFSEE	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	—	These bits are read as 0.	R
5	USMFSEE	Under Switch Minimum Frame Size Error Enable 0: Interrupt disabled 1: Interrupt enabled	R/W ^{*1}
6	TFEE	TAG Filtering Error Enable 0: Interrupt disabled 1: Interrupt enabled	R/W ^{*1}
7	—	This bit is read as 0.	R
15:8	FSEE7 to FSEE0	Frame Size Error Enable n (n = 0 to 7) 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
31:16	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

USMFSEE bit (Under Switch Minimum Frame Size Error Enable)

[Setting condition]

- Writing 1 to this bit.

[Clearing condition]

- Writing 1 to EAEID0.USMFSED register clears this bit.

TFEE bit (TAG Filtering Error Enable)

[Setting condition]

- Writing 1 to this bit.

[Clearing condition]

- Writing 1 to EAEID0.TFED register clears this bit.

FSEEn bits (Frame Size Error Enable n (n = 0 to 7))

[Setting condition]

- Writing 1 to one of these bits.

[Clearing condition]

- Writing 1 to one of EAEID0.FSED bits clears the corresponding bit in this register.

32.3.7.1.3 EAEID0 : Error Interrupt Disable Register 0

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0508

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	FSED7 to FSED0								—	TFED	USMFSED	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
4:0	—	These bits are read as 0.	R
5	USMFSED	Under Switch Minimum Frame Size Error Disable Writing 1 to this bit clears EAEIE0.USMFSEE register.	R
6	TFED	TAG Filtering Error Disable Writing 1 to this bit clears EAEIE0.TFEE register.	R
7	—	This bit is read as 0.	R
15:8	FSED7 to FSED0	Frame Size Error Disable n (n = 0 to 7) Writing 1 to one of these bits clears the corresponding bit in EAEIE0.FSEE register.	R
31:16	—	These bits are read as 0.	R

32.3.7.1.4 EAEIS1 : Error Interrupt Status Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0510

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	TASCTGES	TASGES7 to TASGES0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CULES7 to CULES0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CULES7 to CULES0	CBS n Upper Limit Error Status Flag (n = 0 to 7)	R/W ^{*1}
15:8	—	These bits are read as 0.	R
23:16	TASGES7 to TASGES0	TAS Gate n Error Status Flag (n = 0 to 7)	R/W ^{*1*2}
24	TASCTGES	TAS Cut-Through Gate Error Status Flag	R/W ^{*1*2}
31:25	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

Note 2. This error can occur for the following reasons:

- TAS start time EATASCSTC0/1 is set in the past
- gPTP timer had an offset correction [gPTP]
- The minimum gate time has not been satisfied.

CULESn bits (CBS n Upper Limit Error Status Flag (n = 0 to 7))**[Setting condition]**

- HW: Bit q of this register is set when descriptor queue q corresponding CBS credits has reached its upper limit EACOULMq.CUL.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: Credit calculation stops at EACOULMq.CUL and no further credit are added until frame transmission for the corresponding queue. As a result, the queue throughput is lower than expected.
- SW: This is a software error and should not occur. (Setting TAS with gates that are partially overlapping each other can trigger this error).

TASGESn bits (TAS Gate n Error Status Flag (n = 0 to 7))**[Setting condition]**

- HW: Bit q of this register is set when descriptor queue q corresponding TAS gate had no time to fetch the next gate state value until it starts.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: Gate with restart during next cycle
- SW: If this error occurs frequently, TAS should be reconfigured with a different schedule (this is a software issue because the minimum gate time has not been satisfied). During reconfiguration, this error continues to occur until the new start time is reached and so, it should be ignored.

Caution: This error can occur for the following reasons:

- TAS start time EATASCSTC0/1 is set in the past
- gPTP timer had an offset correction [gPTP]
- The minimum gate time has not been satisfied (a gate time is smaller than 50 ns + EATASHCC.TASJ).

TASCTGES bit (TAS Cut-Through Gate Error Status Flag)**[Setting condition]**

- HW: This register is set when cut-through corresponding TAS gate has no time to fetch the next gate state value until it starts.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to one of these bits.

[Error recovery]

- HW: Gate with restart during next cycle
- SW: If this error occurs frequently, TAS should be reconfigured with a different schedule (this is a software issue because the minimum gate time has not been satisfied). During reconfiguration, this error continues to occur until the new start time is reached and so, it should be ignored.

Caution: This error can occur for the following reasons:

- TAS start time EATASCSTC0/1 is set in the past

- gPTP timer had an offset correction [gPTP]
- The minimum gate time has not been satisfied.

32.3.7.1.5 EAEIE1 : Error Interrupt Enable Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0514

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	TASC TGEE	TASGEE7 to TASGEE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CULEE7 to CULEE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CULEE7 to CULEE0	CBS n Upper Limit Error Enable (n = 0 to 7) 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
15:8	—	These bits are read as 0.	R
23:16	TASGEE7 to TASGEE0	TAS Gate n Error Enable (n = 0 to 7) 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
24	TASCTGEE	TAS Cut-through Gate Error Enable 0: Interrupt disabled 1: Interrupt enabled	R/W ^{*1}
31:25	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

CULEEn bits (CBS n Upper Limit Error Enable (n = 0 to 7))

[Setting condition]

- Writing 1 to one of these bits.

[Clearing condition]

- Writing 1 to one of EAEID1.CULED bits clears the corresponding bit in this register.

TASEEn bits (TAS Gate n Error Enable (n = 0 to 7))

[Setting condition]

- Writing 1 to one of these bits.

[Clearing condition]

- Writing 1 to one of EAEID1.TASGED bits clears the corresponding bit in this register.

TASCTGEE bit (TAS Cut-through Gate Error Enable)

[Setting condition]

- Writing 1 to this bit.

[Clearing condition]

- Writing 1 to EAEID1.TASCTGED register.

32.3.7.1.6 EAEID1 : Error Interrupt Disable Register 1

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0518

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	TASCTGED	TASGED7 to TASGED0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	CULED7 to CULED0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	CULED7 to CULED0	CBS n Upper Limit Error Disable (n = 0 to 7) Writing 1 to one of these bits clears the corresponding bit in EAEIE1.CULEE register.	R
15:8	—	These bits are read as 0.	R
23:16	TASGED7 to TASGED0	TAS Gate n Error Disable (n = 0 to 7) Writing 1 to one of these bits clears the corresponding bit in EAEIE1.TASGEE register.	R
24	TASCTGED	TAS Cut-through Gate Error Disable Writing 1 to this bit clears EAEIE1.TASCTGEE register.	R
31:25	—	These bits are read as 0.	R

32.3.7.1.7 EAEIS2 : Error Interrupt Status Register 2

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0520

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSES7 to DQSES0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	CTDQOES	DQOES7 to DQOES0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOES7 to DQOES0	Descriptor Queue n Overflow Error Status Flag (n = 0 to 7)	R/W ^{*1}
8	CTDQOES	Cut-Through Descriptor Queue Overflow Error Status Flag	R/W ^{*1}
15:9	—	These bits are read as 0.	R
23:16	DQSES7 to DQSES0	Descriptor Queue n Security Error Status Flag (n = 0 to 7)	R/W ^{*1}
31:24	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

DQOESn bits (Descriptor Queue n Overflow Error Status Flag (n = 0 to 7))

[Setting condition]

- HW: Bit q of this register is set.

[ETHA is operating] (EAMC.OPC == 11 and EAMC.OPS == 11)

Descriptor queue is "full (EATDQDCq.DQD == EATDQMq.DNQ)" or "not disabled (EATDQC.TDQDn is not set)" when a descriptor is received for queue q.

[ETHA is not operating] (EAMC.OPC != 11 and EAMC.OPS == 11)

When a descriptor is received for queue q.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to this bit.

[Error recovery]

- HW: Any descriptor received for a full descriptor queue is not accepted by ETHA and is not forwarded to RMAC [RMAC]
- SW: Too many descriptors are received for the corresponding queue. EATDQDCq.DQD setting should be reviewed.

CTDQOES bit (Cut-Through Descriptor Queue Overflow Error Status Flag)

[Setting conditions]

HW: This register has two exclusive set conditions:

- [ETHA is operating] (EAMC.OPC == 11 and EAMC.OPS == 11) Descriptor queue is "full (EACTDQDCq.CTDQD == EACTDQM.CTQDN)" and "not disabled (EATDQC.TCTDQD is not set)" when descriptor is received for cut-through descriptor
- [ETHA is not operating] (EAMC.OPC != 11 or EAMC.OPS != 11) When descriptor is received for cut-through descriptor.

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to this bit.

[Error recovery]

- HW: Any descriptor received for a full descriptor queue is not accepted by ETHA and is not forwarded to RMAC [RMAC]
- SW: Too many descriptors are received for the corresponding queue. EACTDQDC.CTDQD setting should be reviewed.

DQSESn bits (Descriptor Queue n Security Error Status Flag (n = 0 to 7))

[Setting condition]

- HW: Bit q of this register is set when a non-secure descriptor is received (FDESCR.SEC is not set [FWD]) and queue q is a secure queue (EATDQSC.TDQSLn is set).

[Clearing conditions]

- HW: When in RESET mode
- SW: Writing 1 to this bit.

[Error recovery]

- HW: Any non-secure descriptor received for secure descriptor queue is not accepted by ETHA and is not forwarded to CPU
- SW: System-dependent, therefore cannot be defined.

32.3.7.1.8 EAEIE2 : Error Interrupt Enable Register 2

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0524

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSEE7 to DQSEE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	CTDQ OEE	DQOEE7 to DQOEE0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOEE7 to DQOEE0	Descriptor Queue n Overflow Error Enable (n = 0 to 7) 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
8	CTDQOEE	Cut-Through Descriptor Queue Overflow Error Enable 0: Interrupt disabled 1: Interrupt enabled	R/W ^{*1}
15:9	—	These bits are read as 0.	R
23:16	DQSEE7 to DQSEE0	Descriptor Queue n Security Error Enable (n = 0 to 7) 0: Interrupt disabled for descriptor queue i 1: Interrupt enabled for descriptor queue i	R/W ^{*1}
31:24	—	These bits are read as 0.	R

Note 1. Read value differs from written value.

DQOEE_n bits (Descriptor Queue n Overflow Error Enable (n = 0 to 7))

[Setting condition]

- Writing 1 to one of these bits.

[Clearing condition]

- Writing 1 to one of EAEID2.DQOED bits clears the corresponding bit in this register.

CTDQOEE bit (Cut-Through Descriptor Queue Overflow Error Enable)

[Setting condition]

- Writing 1 to this bit.

[Clearing condition]

- Writing 1 to EAEID2.CTDQOED register clears this bit.

DQSEEn bits (Descriptor Queue n Security Error Enable (n = 0 to 7))

[Setting condition]

- Writing 1 to one of these bits.

[Clearing condition]

- Writing 1 to one of EAEID2.DQSED bits clears the corresponding bit in this register.

32.3.7.1.9 EAEID2 : Error Interrupt Disable Register 2

Base address: ETHAm = 0x403C_A000 + 0x2000 × m (m = 0, 1)
 ETHAm_NS = 0x503C_A000 + 0x2000 × m (m = 0, 1)

Offset address: 0x0528

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	DQSED7 to DQSED0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	CTDQ OED	DQOED7 to DQOED0							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	DQOED7 to DQOED0	Descriptor Queue n Overflow Error Disable (n = 0 to 7) Writing 1 to one of these bits clears the corresponding bit in EAEIE2.DQOEE register.	R
8	CTDQOED	Cut-Through Descriptor Queue Overflow Error Disable Writing 1 to this bit clears EAEIE2.CTDQOEE register.	R
15:9	—	These bits are read as 0.	R
23:16	DQSED7 to DQSED0	Descriptor Queue n Security Error Disable (n = 0 to 7) Writing 1 to one of these bits clears the corresponding bit in EAEIE2.DQSEE register.	R
31:24	—	These bits are read as 0.	R

32.4 Register Utilization

32.4.1 Operation Modes

Table 32.4 describes ETHA operation modes.

Table 32.4 ETHA operation modes

Operation mode	GWMS.OPS value	Description
DISABLE	1	- No ongoing transaction - Only status registers are accessible for writing when the agent clock is enabled.
RESET	0	- No ongoing transaction - An internal reset is asserted to reset ETHA logic with the status registers - No register is accessible for writing - RAM values are held.
CONFIG	2	- No ongoing transaction - Static and some dynamic registers are accessible for writing.
OPERATION	3	- Transactions are ongoing - Dynamic registers are accessible for writing.

32.4.1.1 Operation Mode Transitions

Figure 32.2 shows the operating mode transitions.

A mode transition can be triggered by:

- Hardware reset
- Software reset
- Configuration of EAMC.OPC. In this case, mode transition is confirmed by EAMS.OPS.

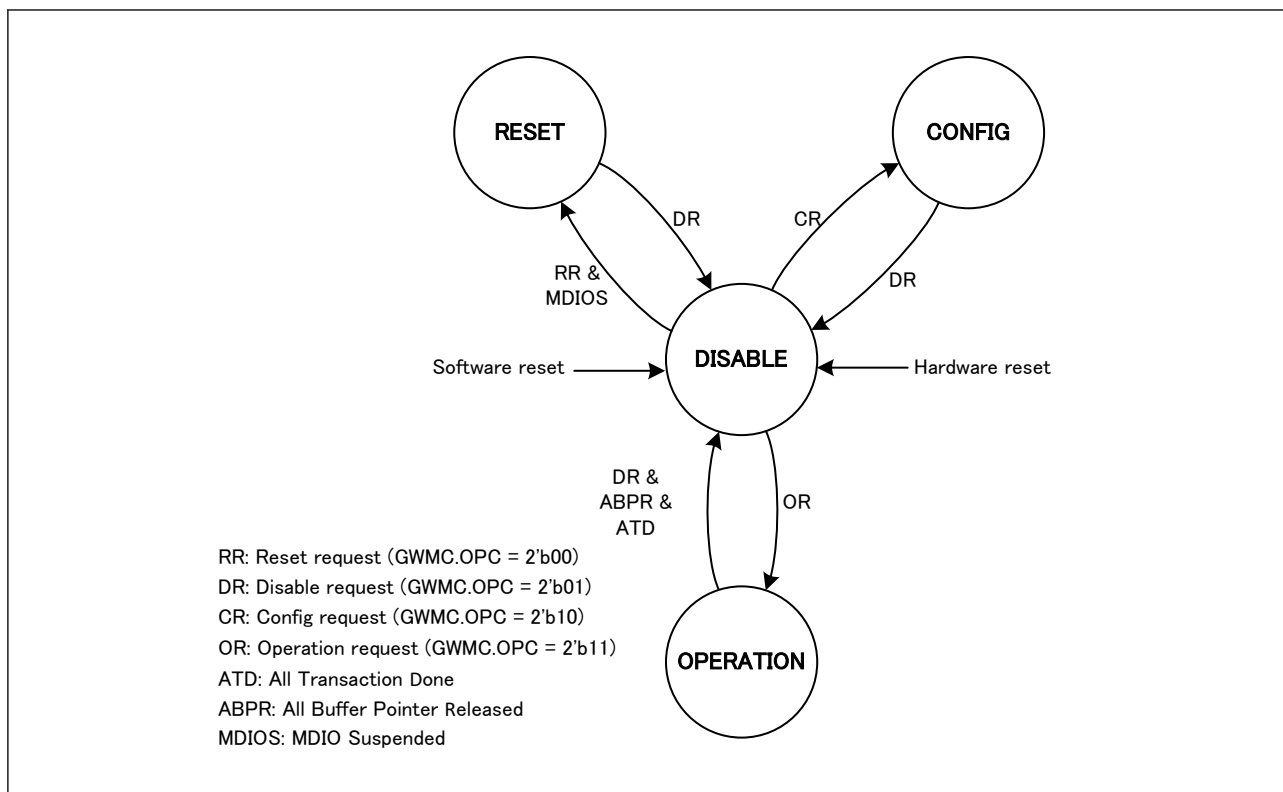


Figure 32.2 **Operating mode transitions**

Complementary information on transition conditions

ABPR:

- All the buffer pointers contained in the ETHA are released to the MFWD.

ATD:

- All the descriptors already received are processed and corresponding frames are transmitted to the Ethernet PHY
- All ongoing reception frames are fully sent to the MFWD.

MDIOS:

- MDIO has no ongoing access.

[Restrictions]

- Software shall only trigger transitions shown in [Figure 32.2](#)
Do not directly transition from OPERATION to CONFIG.

32.4.2 Software Flows

SW: Follow the flows described in this section.

32.4.2.1 Software Flow Legend

[Figure 32.3](#) describes the software flow legend.

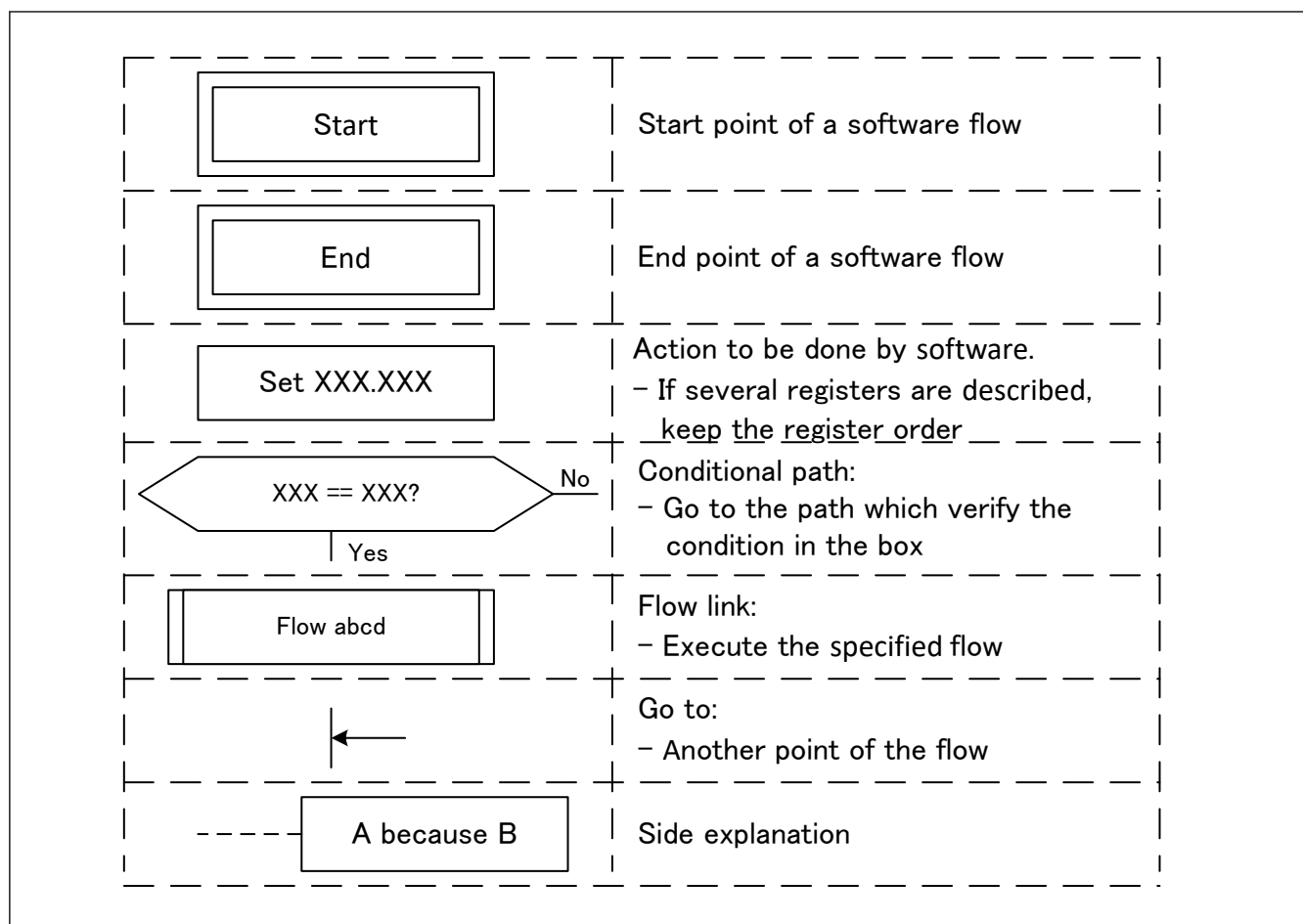


Figure 32.3 Software flow legend

32.4.2.2 Mode Transition Flow

Figure 32.4 describes the mode transition flow.

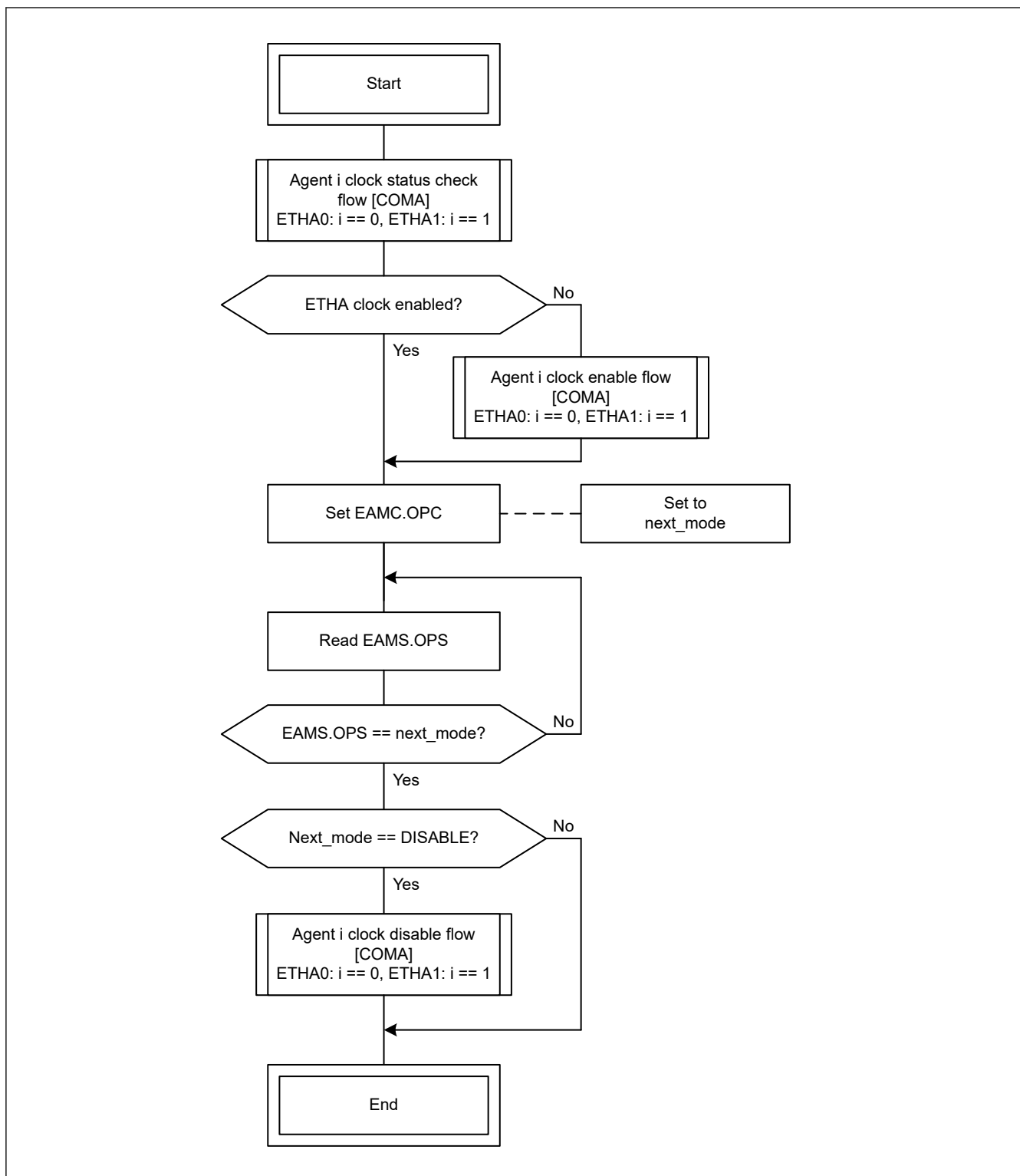


Figure 32.4 Mode transition flow

- Note:
- If software already determines that the clock is enabled, the clock status check step can be skipped.
 - If software does not need to disable the agent clock in DISABLE mode (because it goes directly to another mode or because the clock does not need to be disabled), the clock disable step can be skipped.

32.4.2.3 Reset Flow

Figure 32.5 describes the reset flow.

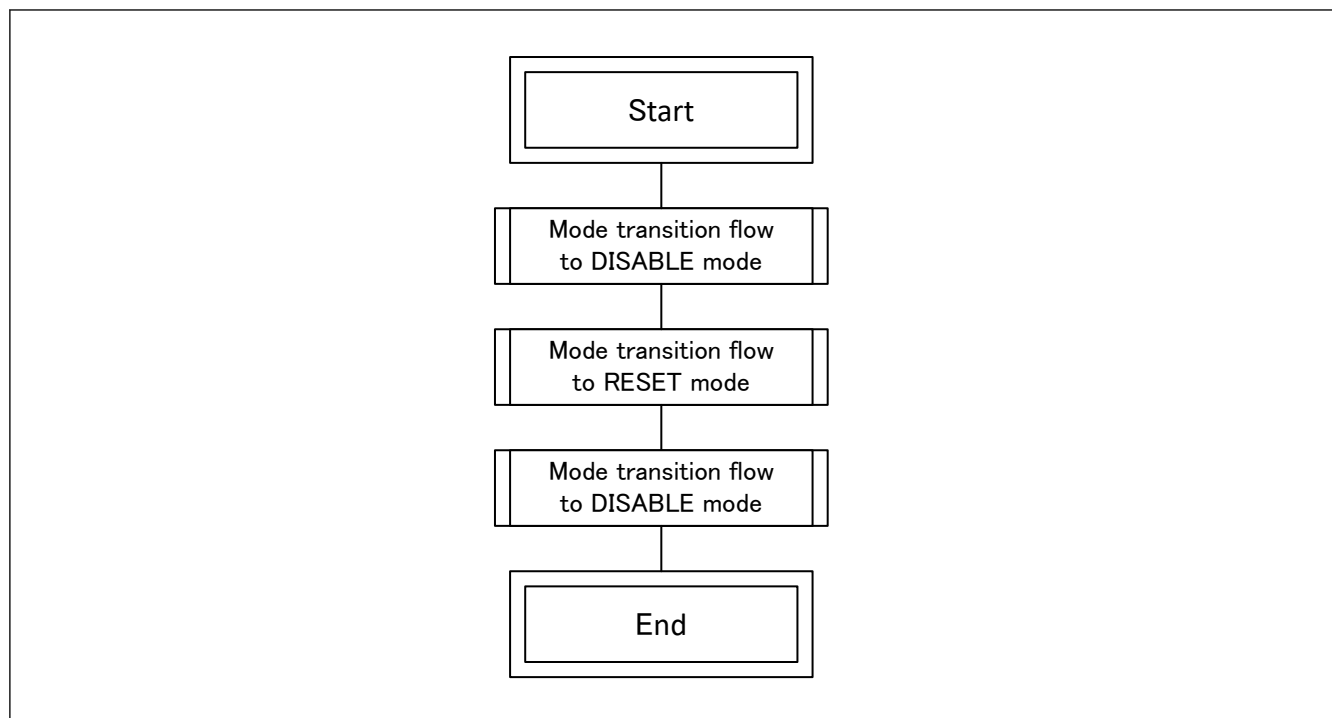


Figure 32.5 Reset flow

32.4.2.4 Initialization Flow

[Figure 32.6](#) describes the initialization flow.

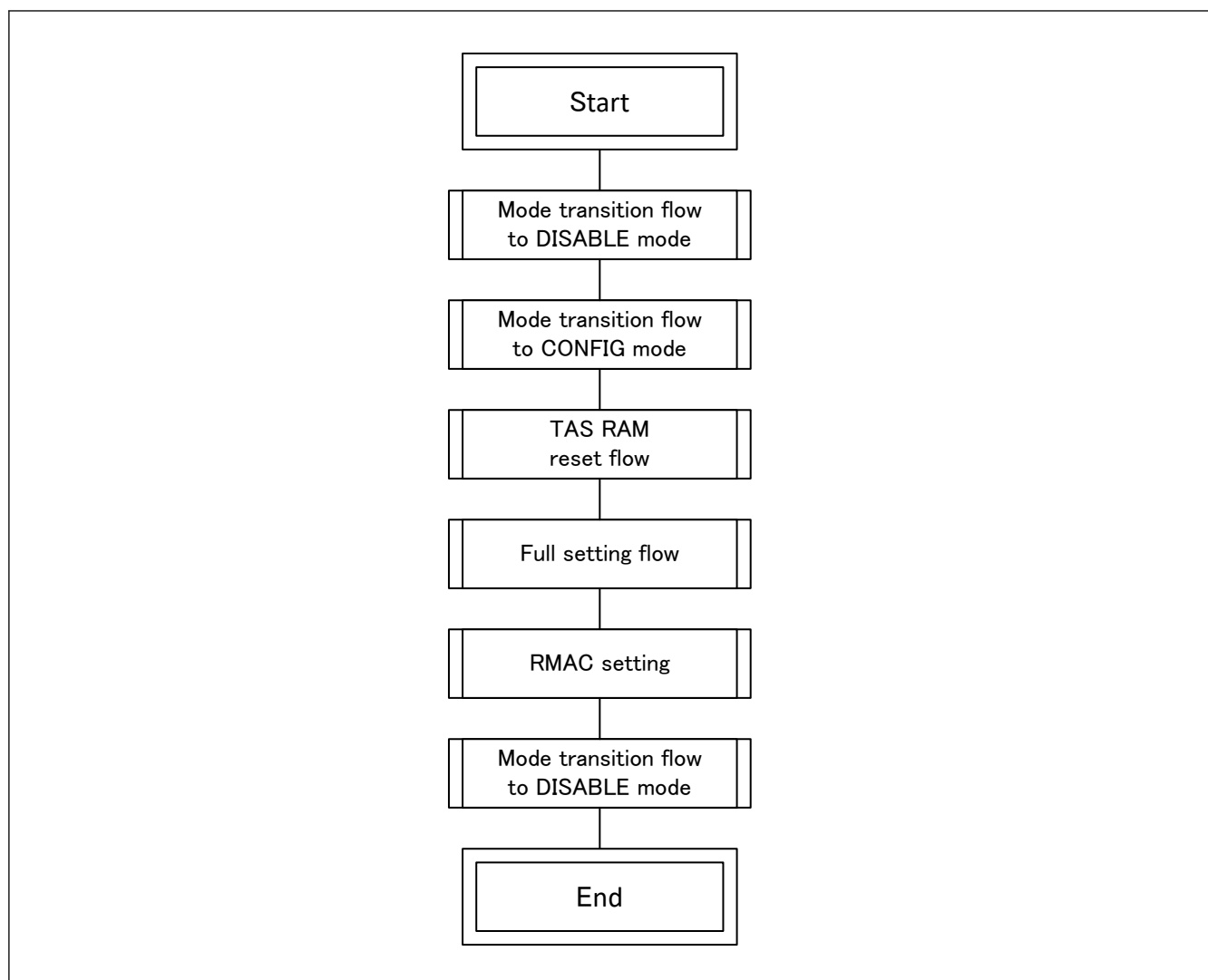


Figure 32.6 Initialization flow

Note: For RMAC setting, see RMAC specification [RMAC].

32.4.2.5 Reinitialization Flow

[Figure 32.7](#) describes the reinitialization flow.

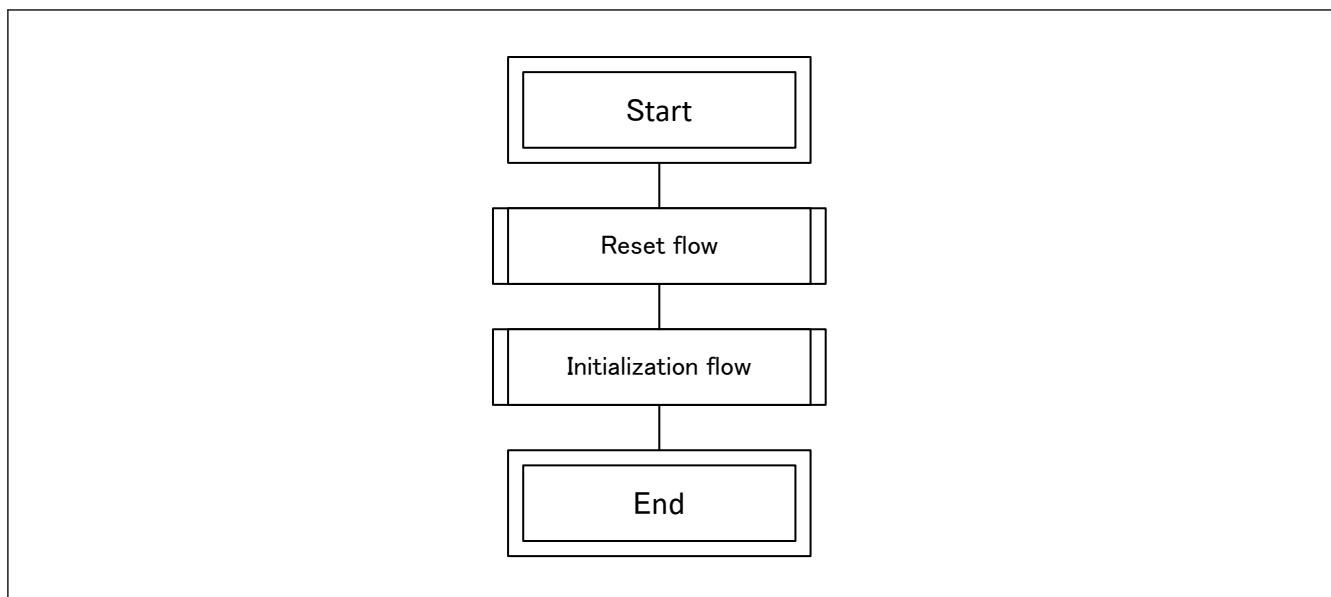


Figure 32.7 Reinitialization flow

32.4.2.6 TAS RAM Reset Flow

Figure 32.8 describes the TAS RAM reset flow.

Note: This flow is not usable in RESET and DISABLE modes.

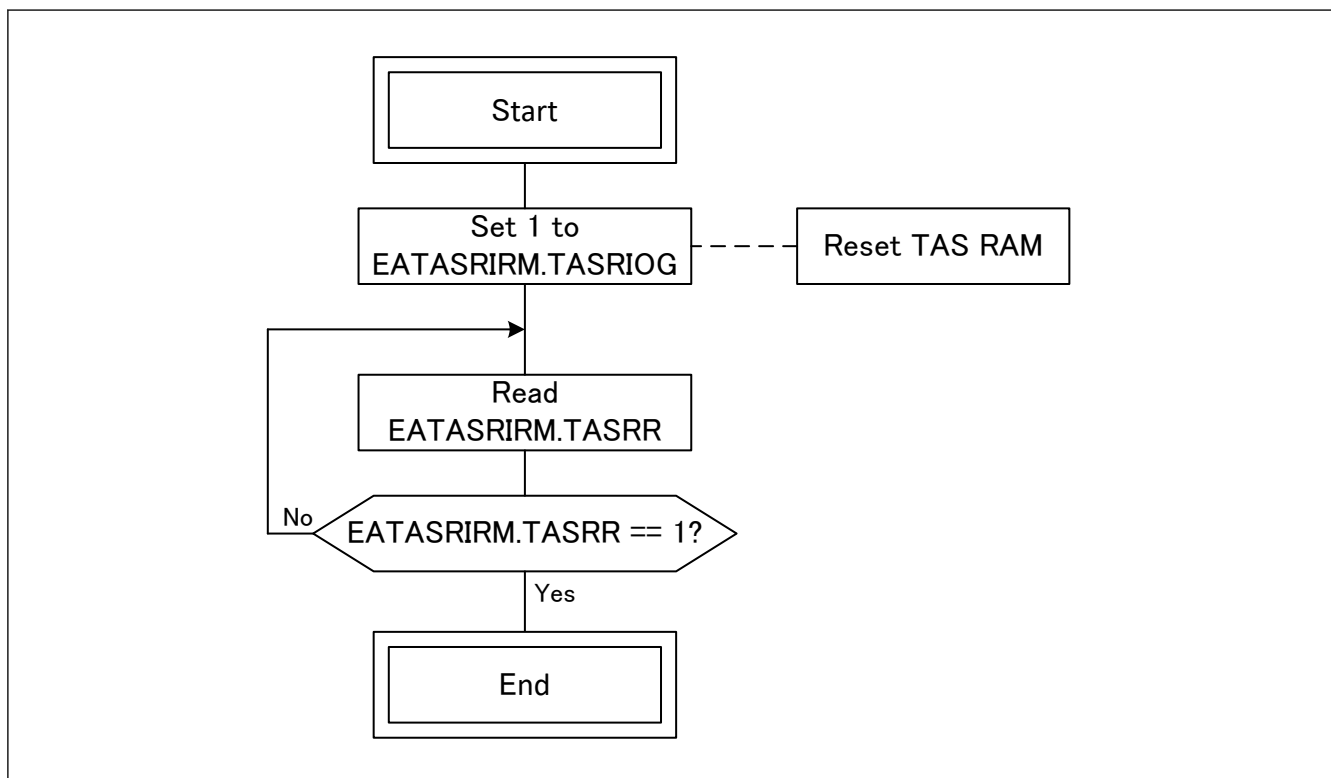


Figure 32.8 TAS RAM reset flow

32.4.2.7 CBS q Setting Flow (q = 0 to 7)

Figure 32.9 describes the CBS q setting flow.

Note: This flow is not usable in RESET and DISABLE modes.

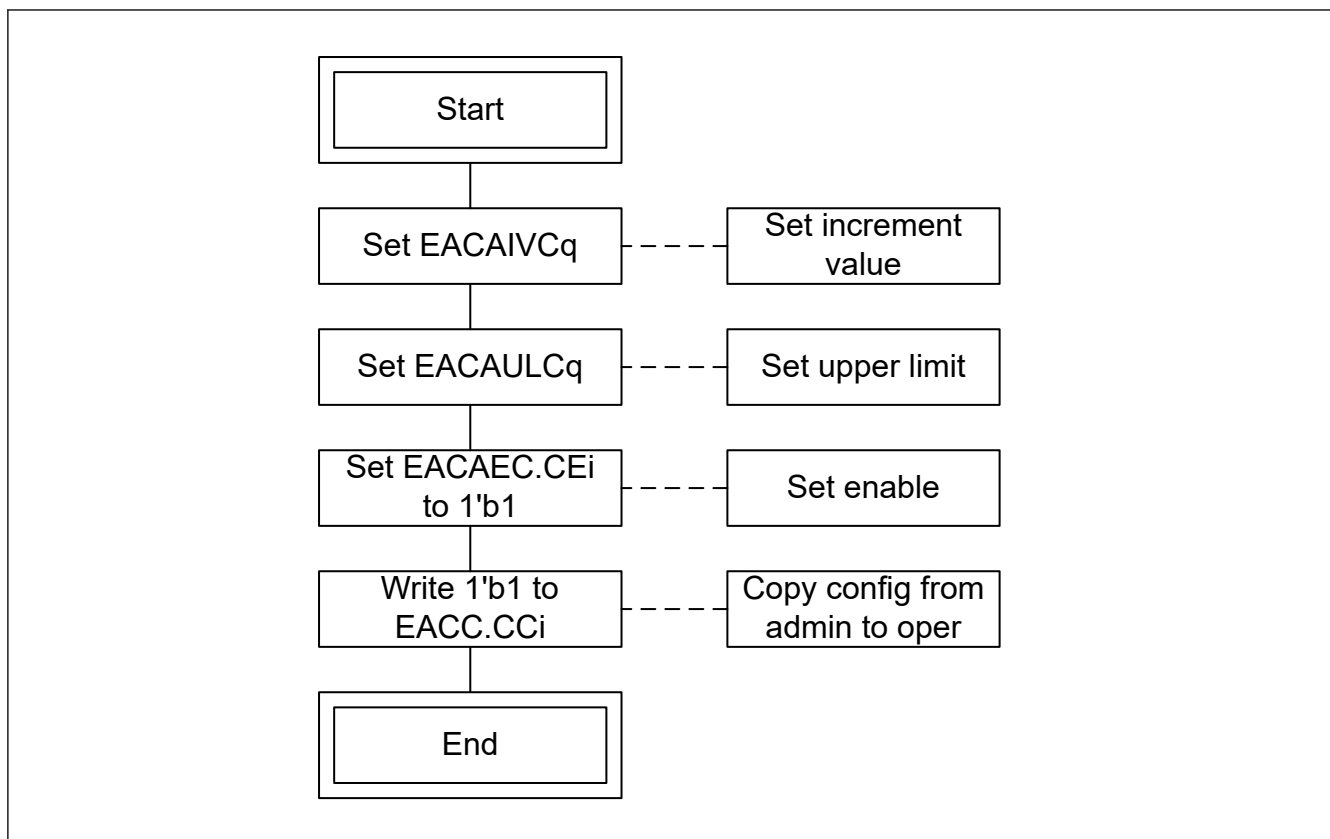


Figure 32.9 CBS q setting flow

32.4.2.8 CBS q Disabling Flow (q = 0 to 7)

Figure 32.10 describes the CBS q disabling flow.

Note: This flow is not usable in RESET and DISABLE modes.

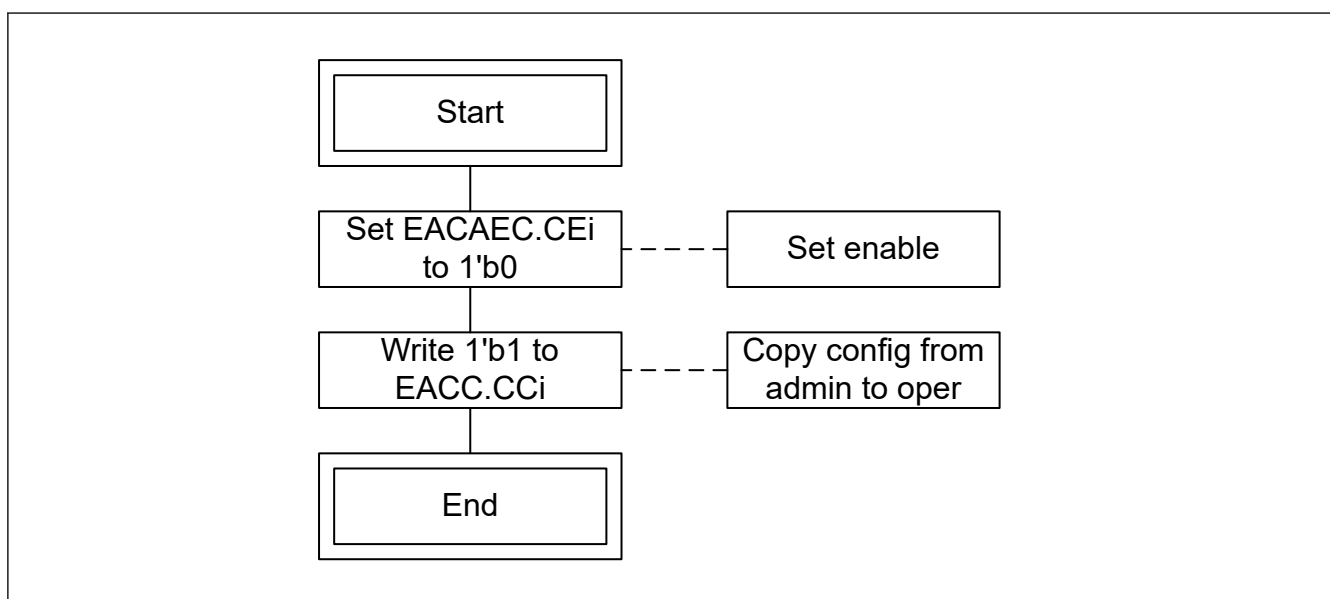


Figure 32.10 CBS q disabling flow

32.4.2.9 TAS Setting Flow

Figure 32.11 describes the TAS setting flow.

Note: This flow is not usable in RESET and DISABLE modes.

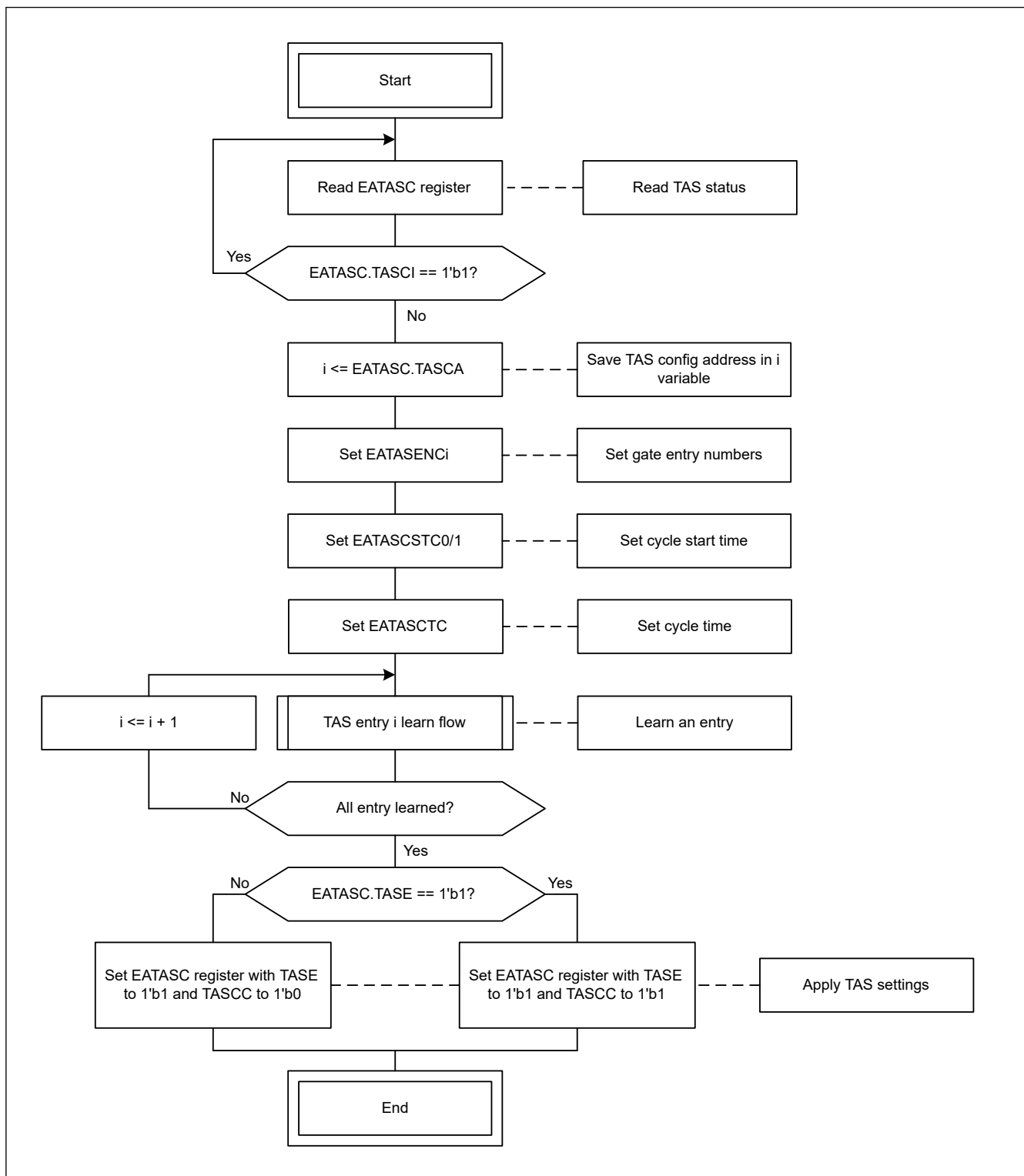


Figure 32.11 TAS setting flow

32.4.2.10 TAS Disabling Flow

Figure 32.12 describes the TAS disabling flow.

Note: This flow is not usable in RESET and DISABLE modes.

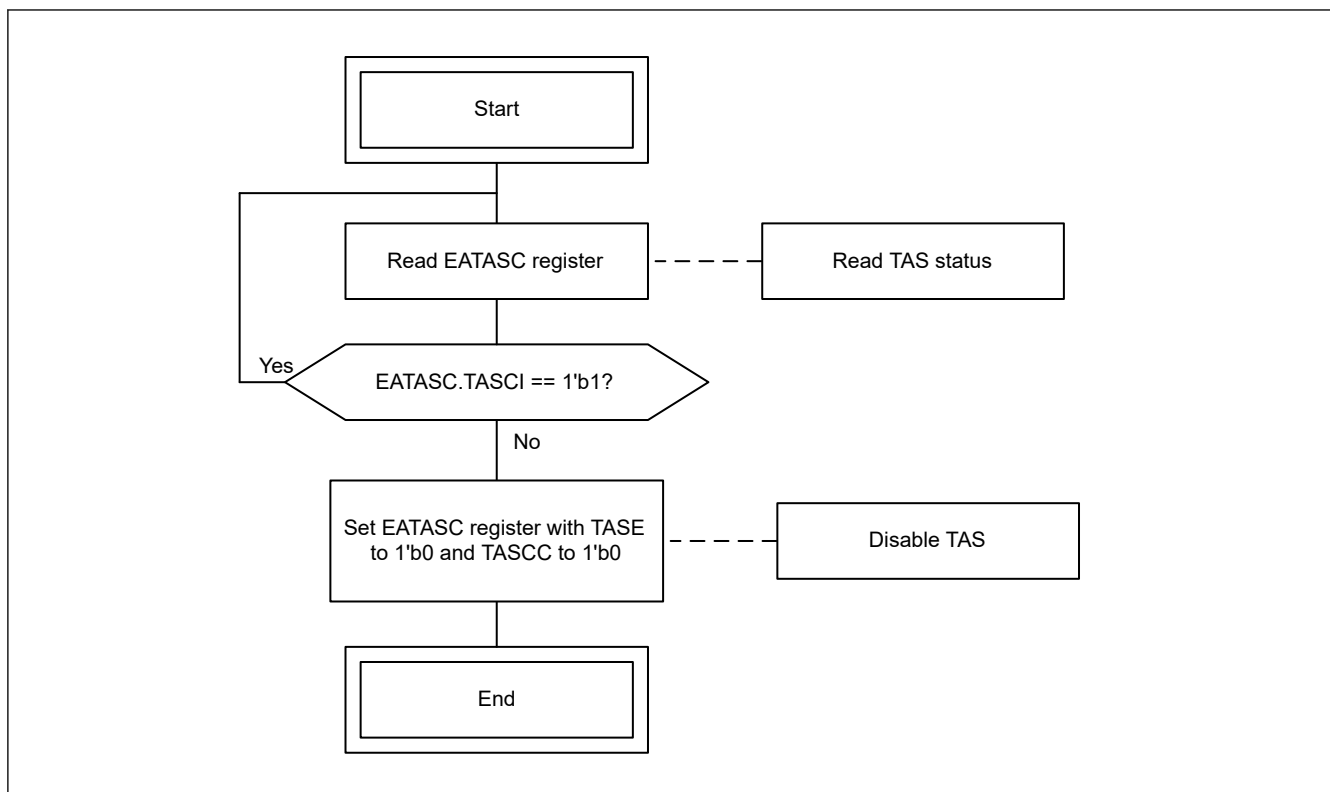


Figure 32.12 TAS disabling flow

32.4.2.11 TAS Enabling Flow

Figure 32.13 describes the TAS enabling flow.

Note: This flow is not usable in RESET and DISABLE modes.

The TAS enabling flow can only be applied after the TAS setting flow has been done at least once. By using the TAS enabling flow, TAS starts with the schedule previously set for the TAS setting flow.

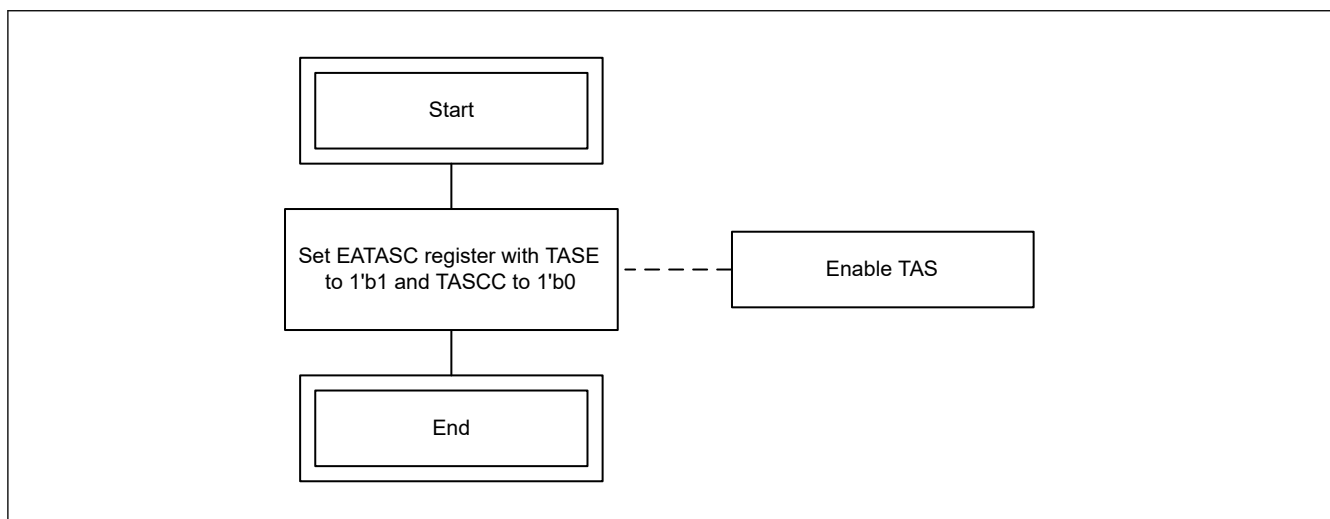


Figure 32.13 TAS enabling flow

32.4.2.12 TAS Entry i Learn Flow

Figure 32.14 describes the TAS entry i learn flow.

Note: This flow is not usable in RESET and DISABLE modes.

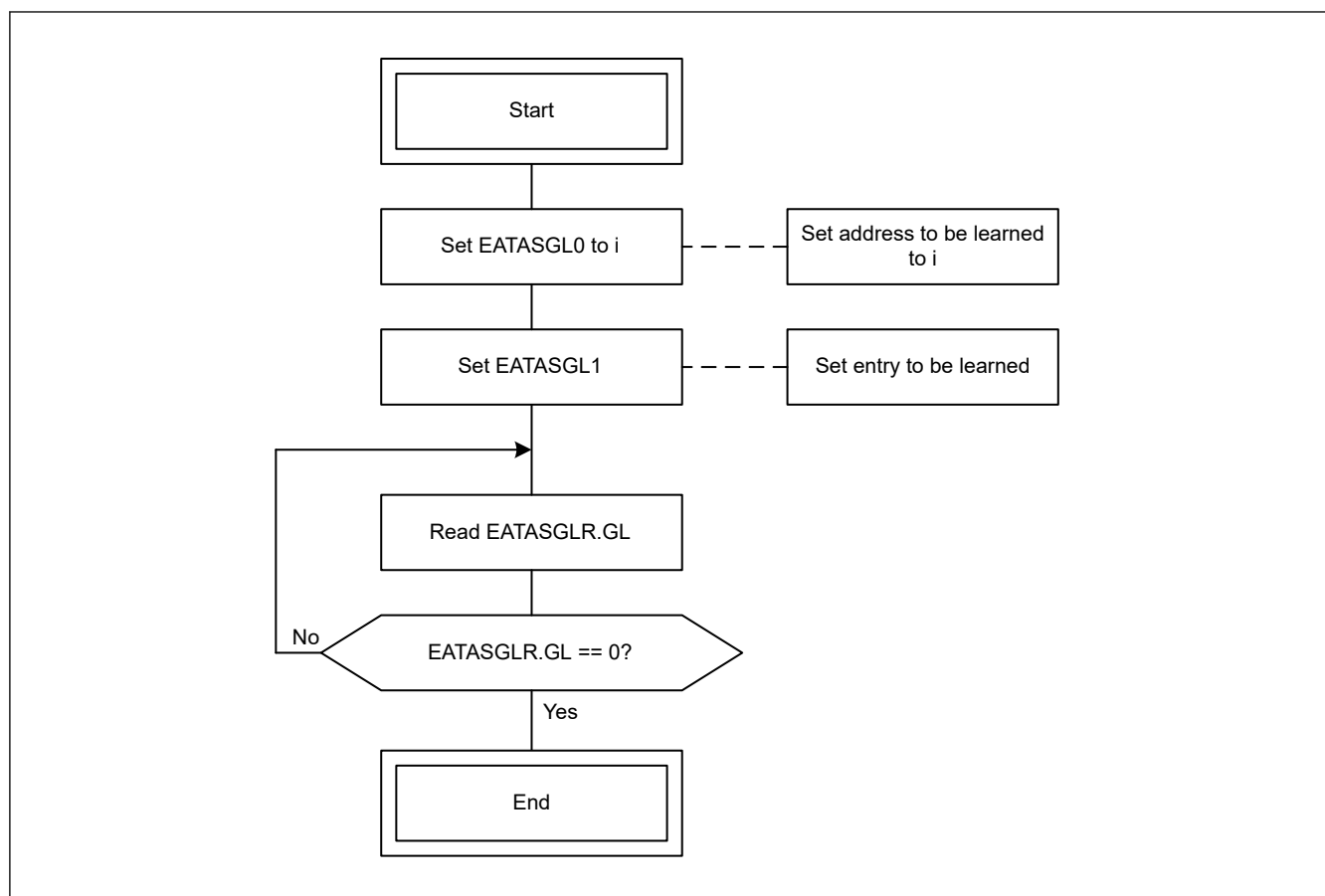


Figure 32.14 TAS entry i learn flow

32.4.2.13 TAS Entry i Read Flow

[Figure 32.15](#) describes the TAS entry i read flow.

Note: This flow is not usable in RESET and DISABLE modes.

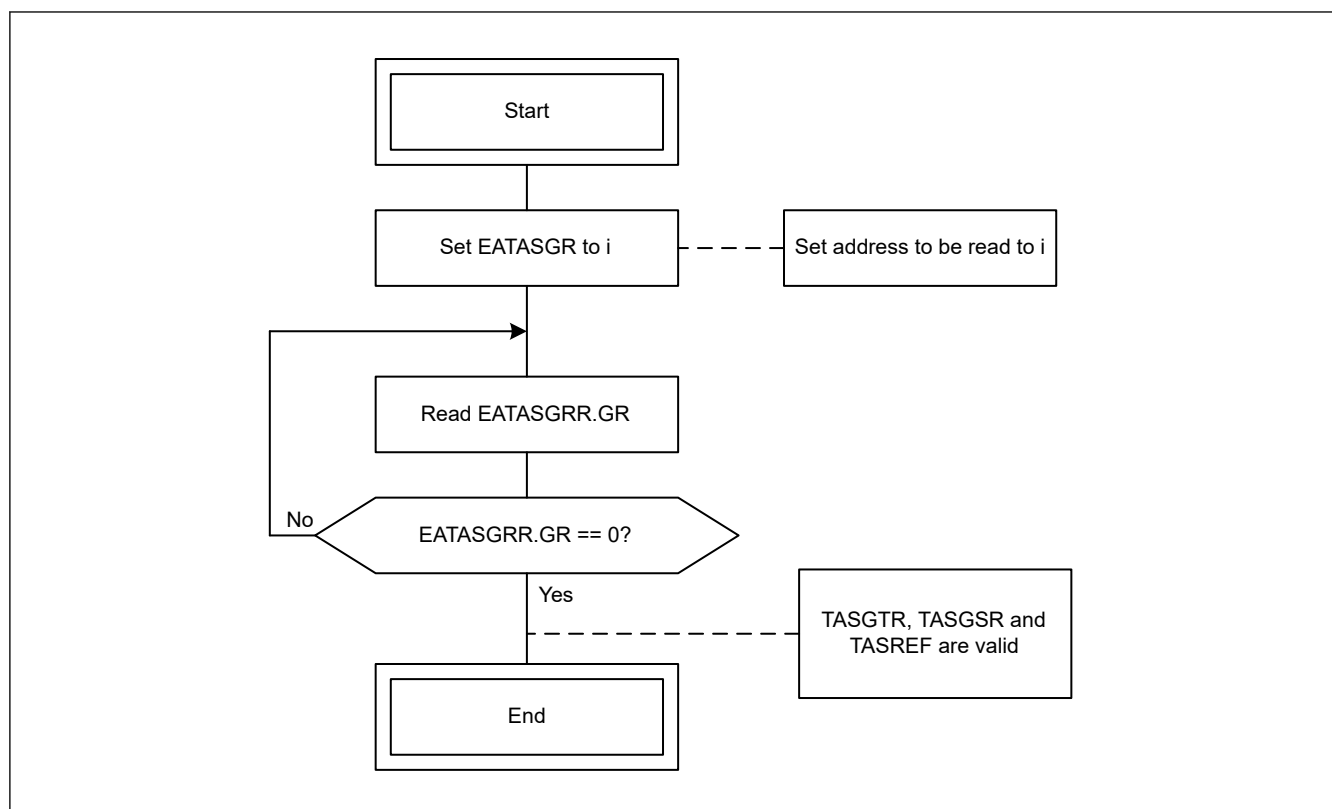


Figure 32.15 TAS entry *i* read flow

32.4.2.14 Interrupt Handling Flow

Figure 32.16 describes the interrupt handling flow.

Note: This flow is not usable in RESET mode.

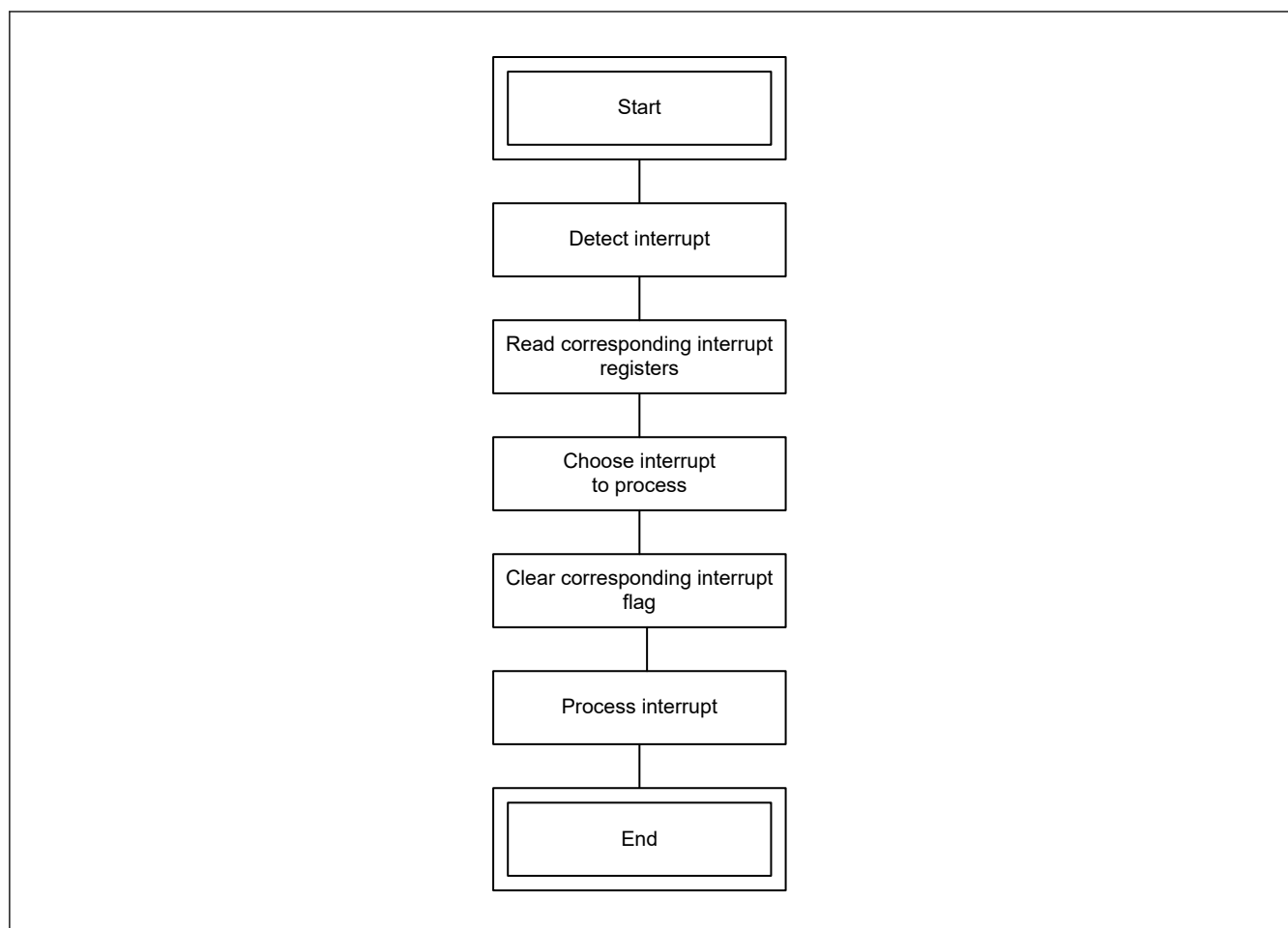


Figure 32.16 Interrupt handling flow

32.4.2.15 Called Software Flows

The flows described in this section can only be called from other flows as indicated in the flow link box in [Figure 32.3](#) and cannot be used alone.

32.4.2.15.1 Full Setting Flow

[Figure 32.17](#) shows the full setting flow.

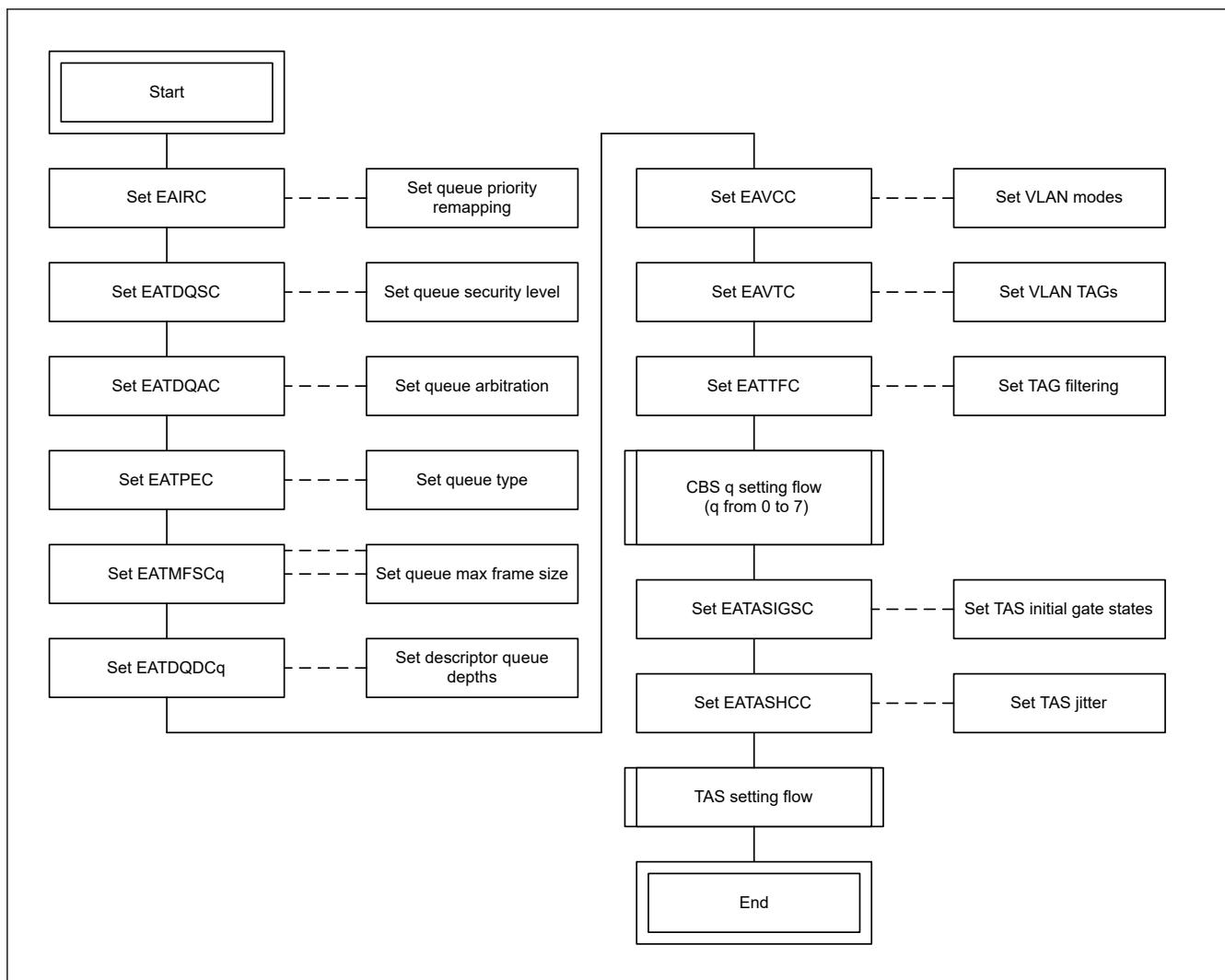


Figure 32.17 Full setting flow

32.4.2.16 Register Writable Without Software Flow

This section details registers that have not been described. These registers can be changed dynamically. However, it is necessary that the initial settings such as the clock enabling have been completed.

32.5 Functional Description

32.5.1 Data Transmission

Ethernet Agent allows data transmission through the ETHA TX data path. [Figure 32.18](#) describes the TX data path.

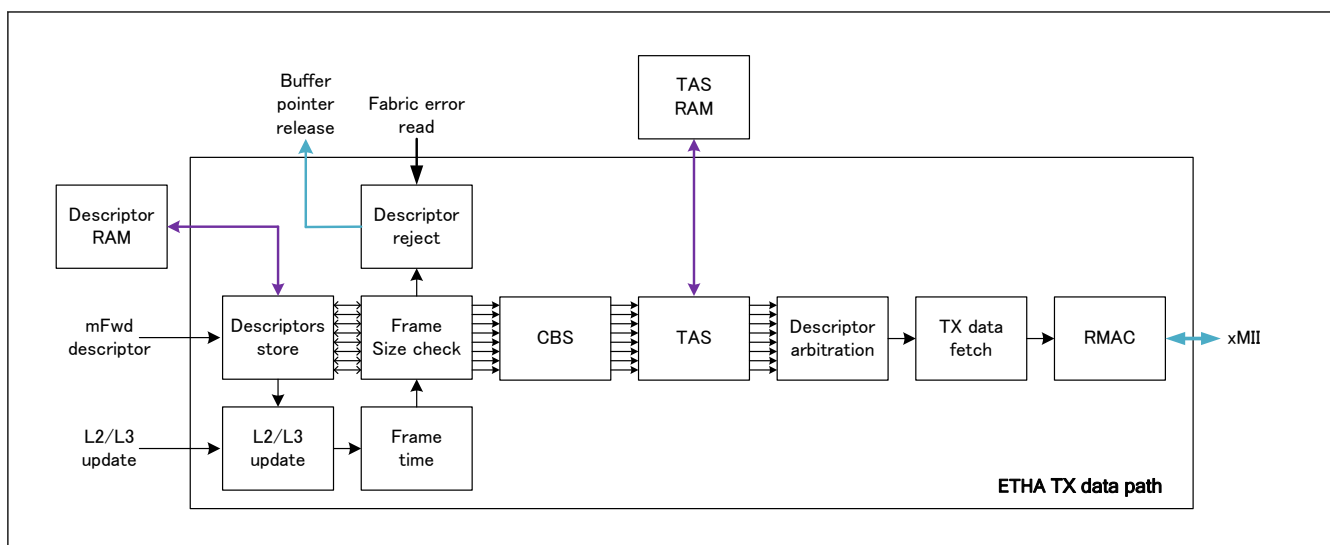


Figure 32.18 ETHA TX data path block diagram

The RX data path is separated in 11 blocks:

- **Descriptor store:** This block aims at storing the descriptor received from the MFWD in the descriptor RAM, reads it and sends it to the L2/L3 update.
- **L2/L3 update:** This block aims at fetching the L2/L3 update rules from the Forwarding Engine to update the frame while sending it.
- **Frame time:** This block aims at calculating how many nanosecond a frame is sent on the ethernet PHY for TAS arbitration. This block exists for TAS hardware purpose and its description is not required for switch utilization so it is not described.
- **Frame size check:** This block checks the frame size and can forward or discard frames.
- **Descriptor reject:** This block aims at releasing the pointer of the frames rejected by the frame size control block. This block exists for switch hardware purpose and its description is not required for switch utilization so it is not described.
- **Cut-through:** This block handles cut-through descriptor coming from MFWD.
- **CBS:** This block aims at shaping the data traffic per descriptor queue by controlling the data throughput per queue [802.1Qav].
- **TAS:** This block aims at shaping the data traffic per descriptor queue by controlling the data transmission with a schedule [802.1Qbv].
- **Descriptor arbitration:** Arbitrates between descriptors based on the descriptor queue using strict priority and/or WRR arbitration.
- **TX data fetch:** This block fetches the frame data from the local RAM, updates it depending on the information obtained by L2/L3 update module and on VLAN control information and releases the frame pointers. The pointer release exists for switch hardware purpose and its description is not required for switch utilization so it is not described.
- **RMAC:** This block handles the data exchange with the Ethernet PHY. The functionalities of this block are described in the RMAC specification [RMAC] so it is not described.

32.5.1.1 Descriptor Store

Descriptor store aims at storing the descriptors coming from the descriptor bus MFWD in the descriptor RAM. This function is the same as the descriptor storage function in the descriptor store block in GWCA. See GWCA specification for more details [GWCA]. Table 32.5 describes the association between ETHA and GWCA registers.

Table 32.5 Descriptor store ETHA/GWCA register association (1 of 2)

Register/signal name in GWCA [GWCA]	Register/signal name in ETHA
GWIRC.IPVRI (i = 0 to 7)	EAIRC.IPVRI (i = 0 to 7)
GWRDQSC.RDQSL	EATDQSC.TDQSL

Table 32.5 Descriptor store ETHA/GWCA register association (2 of 2)

Register/signal name in GWCA [GWCA]	Register/signal name in ETHA
GWRDQC.RDQD	EATDQC.TDQD
GWRDQC.RDQP	EATDQC.TDQP
GWRDQDCq.DQD (q = 0 to 7)	EATDQDCq.DQD (q = 0 to 7)
GWRDQMq.DNQ (q = 0 to 7)	EATDQMq.DNQ (q = 0 to 7)
GWRDQMLMq.DMLQ (q = 0 to 7)	EATDQMLMq.DMLQ (q = 0 to 7)
GWEIS1.DQOES	EAEIS2.DQOES
GWEIS1.DQSES	EAEIS2.DQSES

32.5.1.2 L2/L3 Update

This block is the same as the L2/L3 update block in GWCA. See GWCA specification for more details [GWCA].

32.5.1.3 Frame Size Check

Frame size check aims at checking if a frame has the right properties to be received by CPU using EATMFSCq (q = 0 to 7) registers.

EATMFSCq (q = 0 to 7) registers are used to set the maximum frame size accepted for each descriptor queue. Any frame received for queue q with a size larger than EATMFSCq.MFS is discarded and EAEIS0.FSESq flag is set. The frame size considered is the size of frame on the PHY.

32.5.1.4 Cut-through

Cut-through controls the cut-through descriptors queue to store cut-through descriptors received from MFW using EATDQC.TCTDQD, EACTQC.CTQD and EACTDQDC.CTDQD registers and can be monitored using EACTDQM.CTQDN and EACTDQMLM.CTDMLQ registers.

- EATDQC.TCTDQD register is used to disable the cut-through queue. If the cut-through queue is disabled, eha_ct_ready bit is deasserted, and no descriptor is stored.
- EACTQC.CTQD is used to control the number of clocks that should be waited after receiving a cut-through descriptor to be able to send it for transmission.
- EACTDQDC.CTDQD register is used to set the maximum number of descriptors that can be stored in the cut-through descriptor queue. If the descriptor queue becomes full (EACTDQDC.CTDQD == EACTDQM.CTQDN) eha_ct_ready signal is deasserted. If a descriptor is received for cut-through descriptor queue while it is full, the descriptor is not stored and EAEIS2.CTDQOES flag is set.
- EACTDQM.CTQDN register is used to monitor the current number of descriptors in the cut-through descriptor queue.
- EACTDQMLM.CTDMLQ register is used to monitor the maximum number of descriptors that has been held in cut-through descriptor queue since the previous reset or since the previous time that this has been read.

32.5.1.5 CBS (Credit-based shaper) [802.1Qav]

CBS aims at shaping the data traffic per descriptor queue by controlling the data throughput per queue using EACAEC.CE, EACC.CC, EACAIVCq.CIV (q = 0 to 7) and EACAULCq.CUL (q = 0 to 7) registers and can be monitored using EACOEM.CE, EACOIVMq.CIV (q = 0 to 7), EACOULMq.CUL (q = 0 to 7) and EACGSM.CGS registers and, EAEIS1.CULES interrupt register.

- EACAEC.CE register is used to enable or disable CBS modules.
- EACAIVCq.CIV register is used to configure descriptor queue q CBS throughput.
- EACAULCq.CUL register is used to configure the maximum credit number that can be stored in descriptor queue q CBS module. It is used to monitor associated descriptor queue of CBS module with a long back pressure due to high priority non-CBS-shaped queues interference traffic, or due to queue open time partial overlapping in TAS. If too much back pressure has been applied to a CBS-related descriptor queue, the corresponding EAEIS1.CULES flag is set.
- EACC.CC register is used to apply configurations set in EACAEC.CE, EACC.CC, EACAIVCq.CIV and EACAULCq.CUL registers by copying them to EACOEM.CE, EACOIVMq.CIV and EACOULMq.CUL registers.

- EACOEM.CE register is used to monitor which descriptor queue is used as its CBS module enabled.
- EACOIVMq.CIV register is used to monitor the current throughput set for descriptor queue q.
- EACOULMq.CUL register is used to monitor the current maximum credit number for descriptor queue q.
- EACGSM.CGS register is used to monitor which queue is allowed by CBS to transmit.
- EAEIS1.CULES register is used to flag CBS Upper Limit Errors.

32.5.1.5.1 CBS Setting

CBS setting should follow equation (1) and equation (2). While setting descriptor queue q CBS, CIV should be set to EACAIVCq.CIV register and CUL should be set to EACAULCq.CUL register.

(1) $CIV[byte/cycle] = ((portTransmitRate[bps] / 8) \times (bandwidthFraction[\%] / 100)) / clk_f[Hz] \times (CycleTime / GateOpenTime)$.

For example, when *portTransmitRate* is 1 Gbps, *bandwidthFraction* is 25% and *clk_f* is 200 MHz, CIV is 0.15625. $CIV[19:16] = 0$ $CIV[15:0] = 0.15625 \times 2^{16} = 10,240 = 2800$ H.

(2) $CUL[byte] = (maxInterferenceSize[bit] \times clk_f[Hz] / portTransmitRate[bps] + requestDelay[cycle]) \times CIV[byte/cycle]$.

Where:

- *portTransmitRate* is the ETHA link throughput [RMAC].
- *bandwidthFraction* is the percentage of bandwidth that the CBS corresponding should use.
- *clk_f* is the clock clk frequency.
- $(CycleTime / GateOpenTime)$ is the ratio between the current TAS cycle time (EATASCTM.TASOCT) and the time the CBS corresponding descriptor queue TAS gate is opened during EATASCTM.TASOCT time. When TAS is disabled, $(CycleTime / GateOpenTime)$ is equal to 1.
- *maxInterferenceSize* is the maximum size of any burst of traffic that can delay the transmission of a frame that is available for transmission for this traffic class (see [section 32.5.1.5.2. Maximum Interference Size Calculation](#)).
- *requestDelay* is the time required from descriptor reception to frame transmission ready and is fixed to 50 cycles.

Restrictions:

- SW: All CBS *bandwidthFraction* sum should be smaller than 70% for queues with simultaneously opened TAS gates.

32.5.1.5.2 Maximum Interference Size Calculation

The maximum interface *maxInterferenceSize* size can be calculated for a queue q using equation (3).

Each interference size includes preamble and inter frame gap (IFG).

(3) $maxInterferenceSize = queueInterferenceSizeLow + queueInterfereSize + \sum_i(queueInterferenceSizeHigh[i])$

Where:

- *queueInterferenceSizeLow* is the interference created by all the queues which have a priority smaller than q. This interface size is equal to $(queueLowMaxSize + 20) \times 8$ where *queueLowMaxSize* is **EATMFSCi.MFSi** maximum values for all i smaller than q.
- *queueInterfereSize* is the interference created by queue q and is equal to $(EATMFSCq.MFS + 20) \times 8$
- *queueInterferenceSizeHigh[i]* is the interference created by queue i with a higher priority than q. If CBS is enabled for queue i, *queueInterferenceSizeHigh[i]* is equal to $(EATMFSCi.MFSi + queueLowMaxSize + 40) \times 8$ where *queueLowMaxSize* is **EATMFSCj.MFSj** maximum values for all j smaller than i. If CBS is disabled for queue i, *queueInterferenceSizeHigh[i]* is equal to $(MaxFrameBurst \times EATMFSCi.MFSi + queueLowMaxSize + 40) \times 8$ where *queueLowMaxSize* is **EATMFSCj.MFSj** maximum values for all j smaller than i and where *MaxFrameBurst* is the maximum number of frames that can be expected in a burst. *MaxFrameBurst* is a variable which is system-dependent and cannot be defined.

32.5.1.6 TAS (Time Aware Shaper) [802.1Qbv]

TAS aims at shaping the data traffic per descriptor queue by controlling a AVTP based schedule using EATASC, EATASIGSC, EATASENCi (i = 0 to 7), EATASCTENC, EATASCSTC0/1, EATASCTC, EATASGL0/1,

EATASGLR, EATASHCC registers and can be monitored using ENTASEN*Mi* ($q = 0$ to 7), EATASCTEN*M*, EATASCSTM0/1, EATASCTM, EATASGR, EATASGRR, EATASRIRM, EATASSM registers and, EAIEIS0.TASCTGEES, EAIEIS1.TASGES and EAIEIS1.TASCTGES interrupt registers.

- EATASC register is used for TAS configuration and enabling.
- EATASIGSC register is used to set the initial gate states.
- EATASENC*i* register is used to set the number of entries used in TAS RAM by gate *i*.
- EATASCTENC register is used to set the number of entries used in TAS RAM by cut-through gate.
- EATASCSTC0/1 registers are used to set the cycle start time.
- EATASCTC register is used to set the cycle time.
- EATASGL0/1 and EATASGLR registers are used to write entries in the TAS RAM.
- EATASHCC is used to set the TAS jitter caused by the switch architecture. ENTASEN*Mi* register is used to monitor the number of entries used in the TAS RAM by gate *i* for the ongoing schedule.
- EATASCTEN*M* register is used to monitor the number of entries used in the TAS RAM by cut-through gate for the ongoing schedule.
- EATASCSTM0/1 registers are used to monitor the next cycle start time for the ongoing schedule.
- EATASCTM register is used to monitor the cycle time for the ongoing schedule.
- EATASGR and EATASGRR registers are used to read entries in the TAS RAM.
- EATASRIRM register is used to initialize TAS RAM entries to all 0s.
- EATASSM register is used to monitor TAS operation.
- EAIEIS1.TASGES and EAIEIS1.TASCTGES registers are used to monitor gates errors due to a lack of time to read the TAS RAM.

32.5.1.6.1 TAS General Behavior

TAS module is a scheduler which allows frames to be transmitted depending on their descriptor queue and time by allowing frame transmission when its corresponding gate is opened. TAS scheduler uses a time reference of 64-bit AVTP timer (derived from gPTP timer [gPTP]).

TAS general behavior can be summed up in the following four categories and is described in this section. The TAS schedule is described in section 8.1.6.2.

- TAS enabling
- TAS configuration change
- TAS disabling
- gPTP offset correction recovery [gPTP].

Note: HW: TAS module does not support 64-bit AVTP timer overflows.

1. TAS enabling

TAS enabling happens when TAS is set while being disabled (EATASC.TASE not set).

(a) TAS enabling in the future

TAS module can be such that its schedule starts in the future. [Figure 32.19](#) describes TAS behavior when its schedule starts in the future. The schedule used in this figure is just an example and does not correspond to anything relevant for TAS general behavior explanation.

Restrictions:

- SW: To enable TAS in the future, ensure that EATASC.TASE is set before the AVTP timer reaches the following time: $\{EATASCSTC1, EATASCSTC0\} - 2 \times EATASCTC.TASACT$.

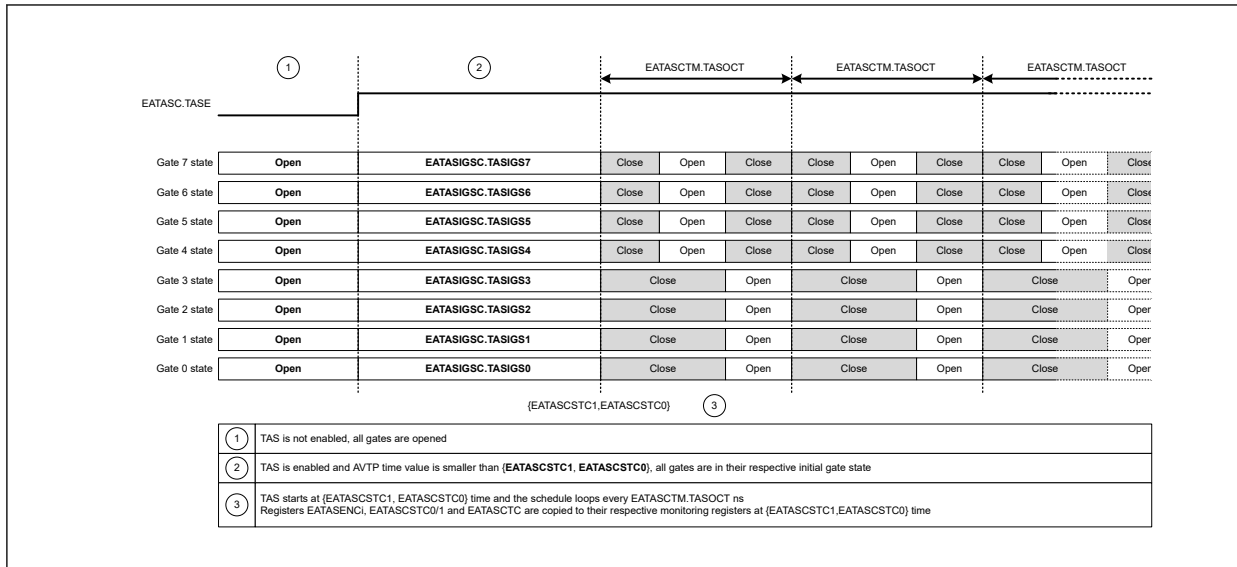


Figure 32.19

TAS behavior when its schedule starts in the future

Restrictions:

- HW: When EATASC.TASE is set, an ongoing frame crosses a gate close state if its initial gate state is closed
- HW: If EATASC.TASE is set such that an ongoing frame does not have the time to finish transmitting before {EATASCSTC1, EATASCSTC0} is reached, the ongoing frame can cross a gate close state in the schedule first cycle
- HW: If EATASC.TASE is set between {EATASCSTC1, EATASCSTC0} - 2 × EATASCTC.TASACT and {EATASCSTC1, EATASCSTC0}, there is a possibility that enabling cannot happen properly and TAS module recovers during 2 × cycle_time[ns] from the start of the new cycle where cycle_time is the schedule cycle time set in EATASCTC.

(b) TAS enabling in the past

TAS module can be such that its schedule should have started in the past. Figure 32.20 describes TAS behavior when its schedule should have started in the past. The schedule used in this figure is just an example and does not correspond to anything relevant for TAS general behavior explanation.

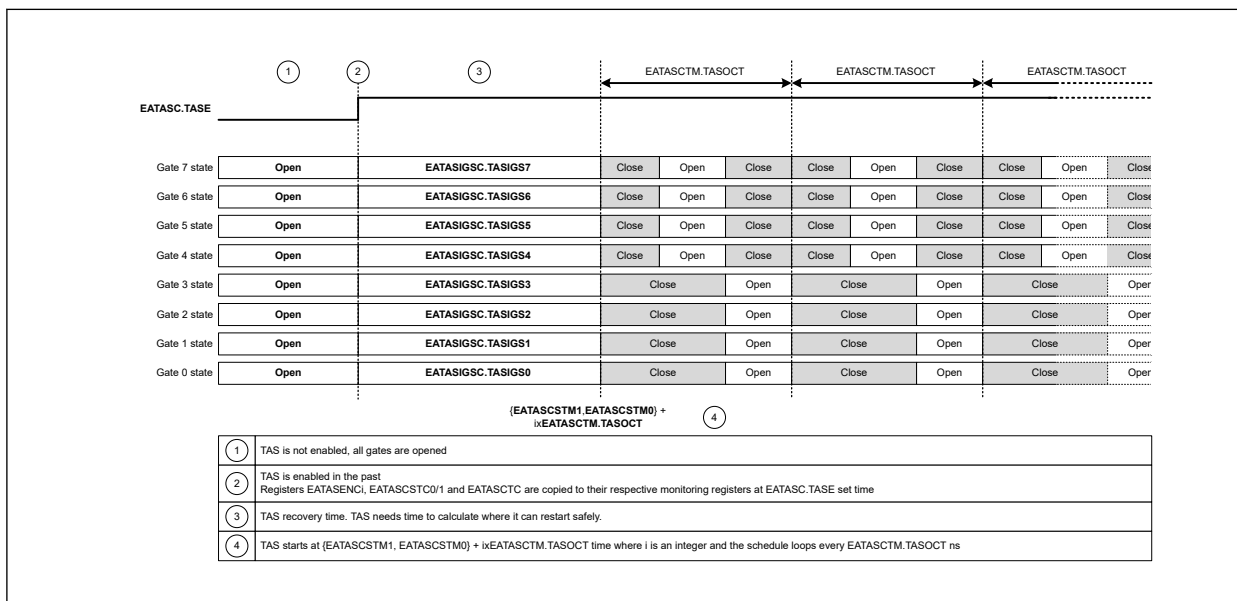


Figure 32.20

TAS behavior when its schedule should have started in the past

TAS recovery time (*tas_recovery_time*) maximum value can be calculated by equation (4).

$$(4) \text{ tas_recovery_time[ns]} = \text{clk_period[ns]} \times (\text{tase_set_time[ns]} - \text{start_time[ns]}) / \text{cycle_time[ns]} + 2 \times \text{cycle_time[ns]}$$

Where:

- *clk_period* is the clock clk period
- *tase_set_time* is the AVTP timer time at which the EATASC.TASE has been set
- *start_time* is the AVTP timer time at which the schedule has been set to start ({EATASCSTC1, EATASCSTC0})
- *cycle_time* is the schedule cycle time set in EATASCTC.

Functions:

- Equation (4) comes from the fact that TAS switches its start time by *cycle_time* every clock during recovery. A problem resulting from that is that the recovery time increases proportionally with *tase_set_time[ns] - start_time[ns]*. To avoid this issue, TAS can switch its start time by $256^i \times \text{cycle_time}$ ($i = 0$ to 6) every clock when start time is too far in the past.

Restrictions:

- HW: Because of the start setting in the past, the hardware guard band cannot guarantee from EATASC.TASE setting time to the end of the first cycle (frames can cross a gate close state)
- HW: Because of the start setting in the past, EAEIS1.TASGES gate errors can be set.

2. TAS configuration change

TAS configuration change happens when TAS is set while being enabled (EATASC.TASE already set).

(a) TAS configuration change in the future

TAS module can be such that its schedule is reconfigured in the future. Figure 32.21 describes TAS behavior when its schedule is reconfigured in the future.

The schedule used in this figure is just an example and does not correspond to anything relevant for TAS general behavior explanation.

Restrictions:

- SW: To enable TAS in the future, ensure that EATASC.TASCC is set before the AVTP timer reaches the following time: $\{\text{EATASCSTC1, EATASCSTC0}\} - 2 \times \text{EATASCTC.TASACT}$.

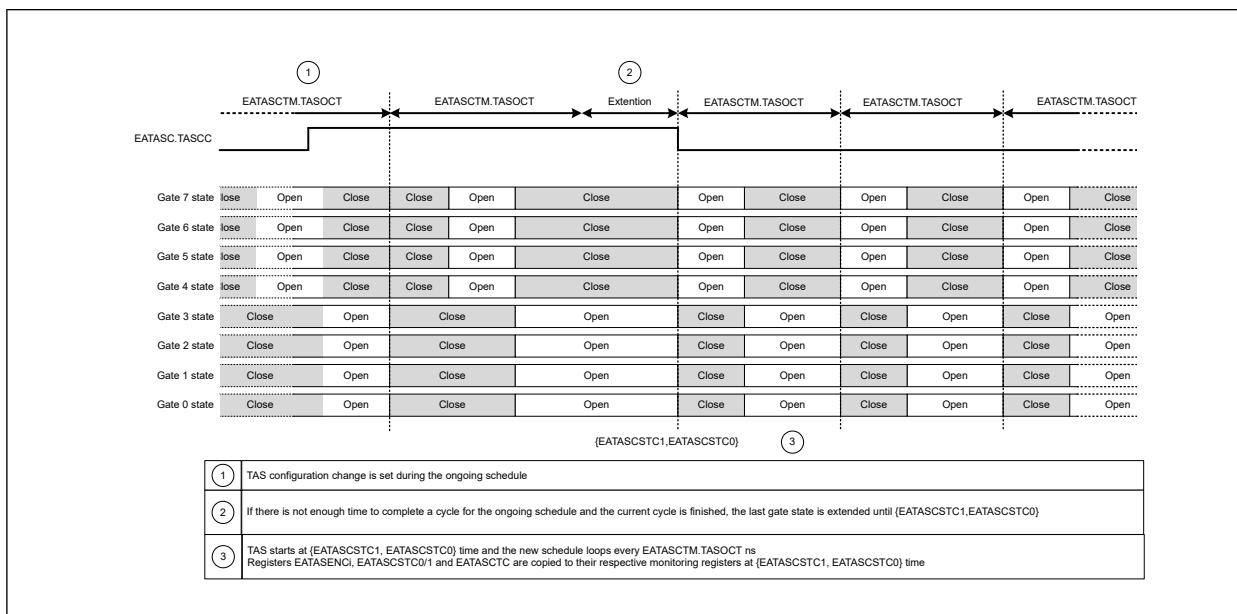


Figure 32.21

TAS behavior when its schedule is reconfigured in the future

Restrictions:

- HW: If EATASC.TASCC is set between $\{\text{EATASCSTC1, EATASCSTC0}\} - 2 \times \text{EATASCTC.TASACT}$ and $\{\text{EATASCSTC1, EATASCSTC0}\}$ there is a possibility that configuration change cannot happen properly and

TAS module recovers during $2 \times \text{cycle_time[ns]}$ from the start of the new cycle where cycle_time is the schedule cycle time set in EATASCTC.

(b) TAS configuration change in the past

TAS module can be such that its schedule reconfiguration should have happened in the past. Figure 32.22 describes TAS behavior when its schedule reconfiguration should have happened in the past. The schedule used in this figure is just an example and does not correspond to anything relevant for TAS general behavior explanation.

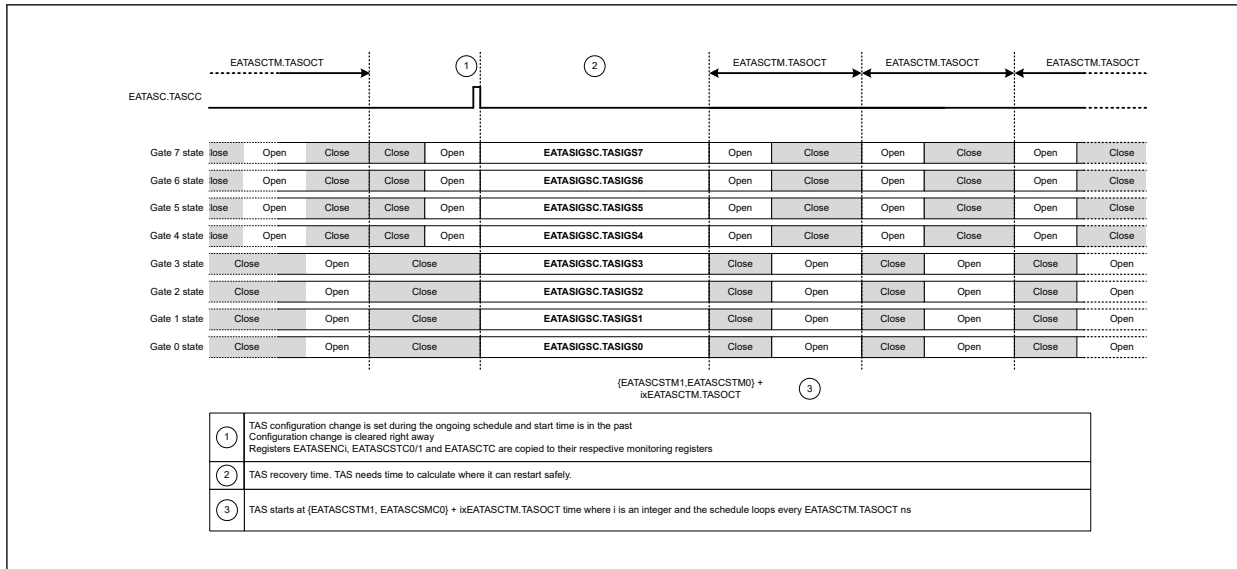


Figure 32.22

TAS behavior when its schedule reconfiguration should have happened in the past

TAS recovery time (tas_recovery_time) maximum value can be calculated by equation (5).

$$(5) \text{ tas_recovery_time[ns]} = \text{clk_period[ns]} \times (\text{tascc_set_time[ns]} - \text{restart_time[ns]}) / \text{cycle_time[ns]} + 2 \times \text{cycle_time[ns]}$$

Where:

- clk_period is the clock clk period
- tascc_set_time is the AVTP timer time at which the EATASCTM.TASCC has been set
- restart_time is the AVTP timer time at which the schedule has been set to restart ($\{\text{EATASCTC1}, \text{EATASCTC0}\}$)
- cycle_time is the schedule cycle time set in EATASCTC.

Functions:

- Equation (5) comes from the fact that TAS switches its start time by cycle_time every clock during recovery. A problem resulting from that is that the recovery time increases proportionally with $\text{tas_set_time[ns]} - \text{start_time[ns]}$. To avoid this issue, TAS can switch its start time by $256^i \times \text{cycle_time}$ ($i = 0$ to 6) every clock when start time is too far in the past.

Restrictions:

- HW: Because of the start setting in the past, the hardware guard band cannot guarantee from EATASCTM.TASCC setting time to the end of the first new cycle (frames can cross a gate close state)
- HW: Because of the start setting in the past, EAEIS1.TASGES gate errors can be set.

3. TAS disabling

TAS module can be disabled during run-time. Figure 32.23 describes TAS behavior when its schedule is disabled. The schedule used in this figure is just an example and does not correspond to anything relevant for TAS general behavior explanation.

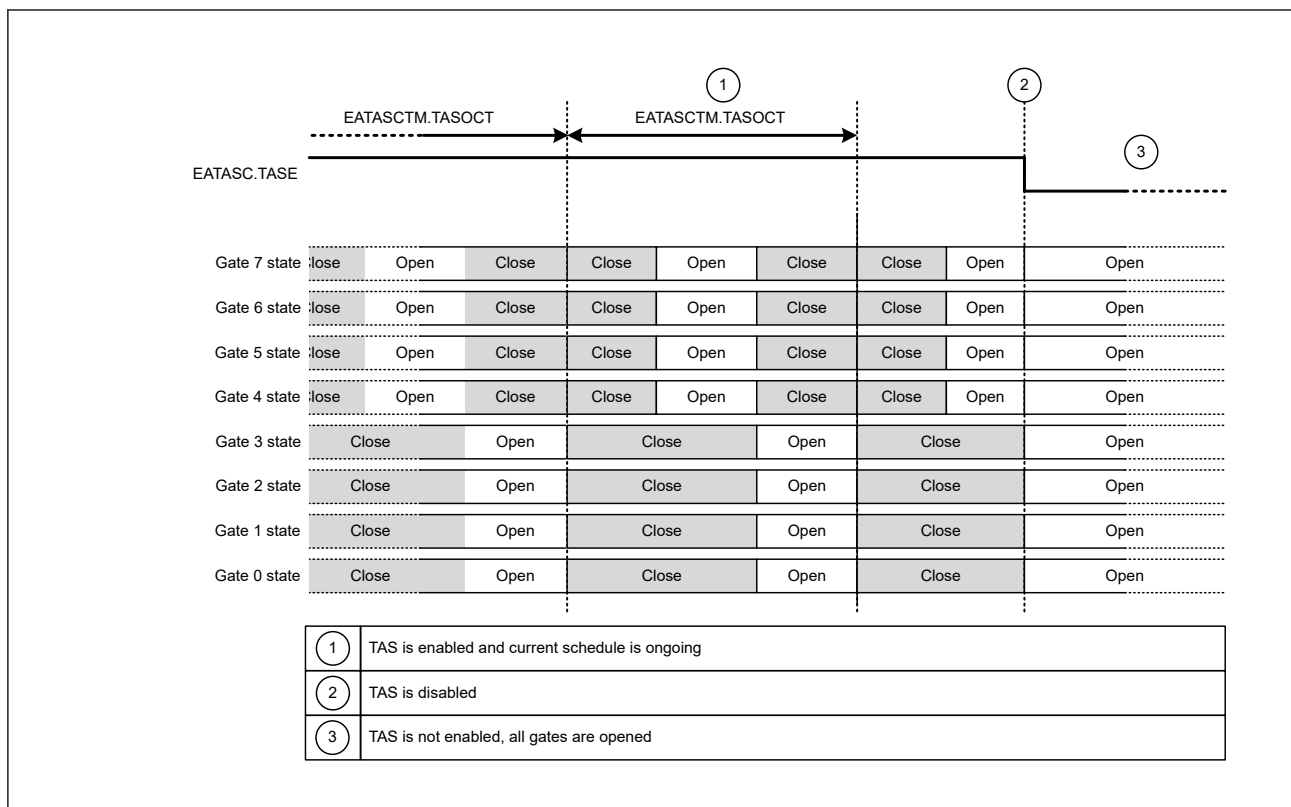
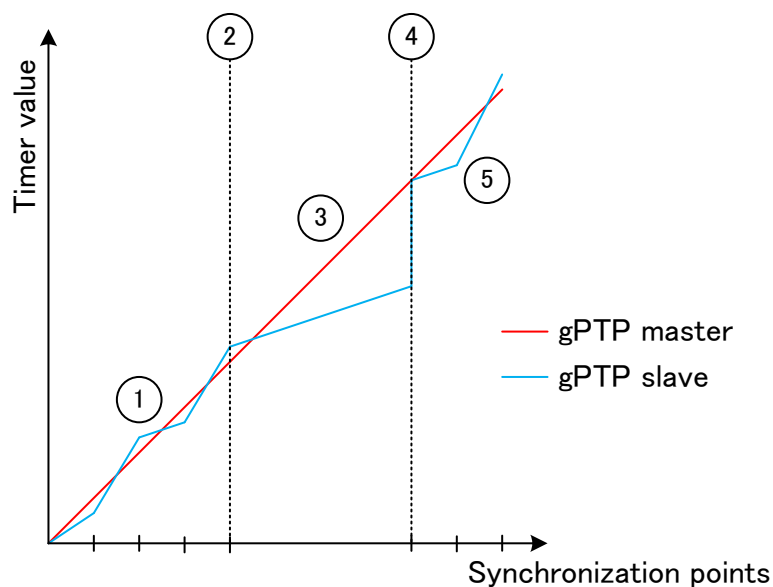


Figure 32.23 TAS behavior when its schedule is disabled

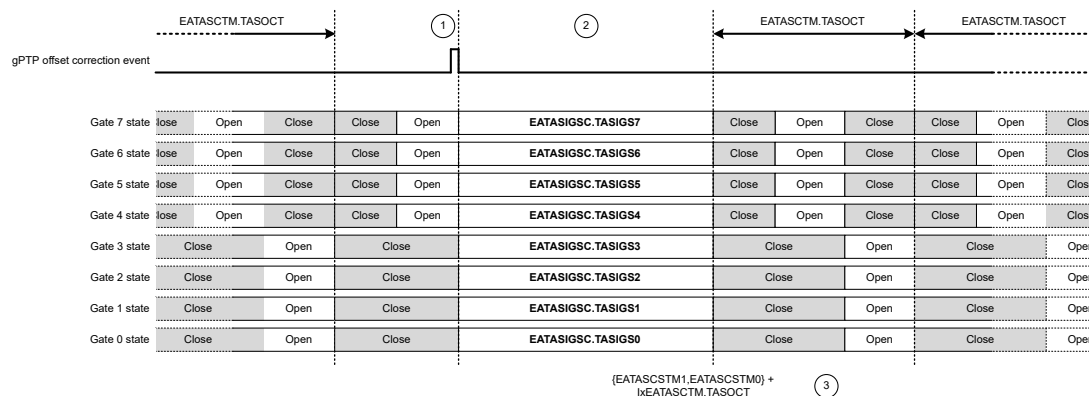
4. gPTP offset correction recovery [gPTP]

gPTP timer control loop in normal condition corrects the gPTP timer using the gPTP incremental value. In this case the gPTP timer should be incremented for every clock by *clk_period* ns (clock clk period) for TAS to operate normally. However, when the gPTP timer is deviating from its corresponding master timer value, software might correct the gPTP offset value instead of the incremental value. Therefore, the gPTP timer and the AVTP timer can increase in time as described in [Figure 32.24](#). In this case, TAS cannot operate normally and must recover as described in [Figure 32.25](#).



①	The slave is synchronized to the master
②	Last successful synchronization point
③	Synchronization is lost. The gPTP slave is deriving
④	Synchronization is done successfully, the gPTP slave offset gets corrected.
⑤	The slave is synchronized to the master

Figure 32.24 gPTP offset correction



①	gPTP offset is corrected by the software so gPTP timer jumps in time
②	TAS recovery time. TAS needs time to calculate where it can restart safely.
③	TAS resynchronizes at $(\text{EATASCTM1}, \text{EATASCTM0}) + \text{I} \times \text{EATASCTM.TASOCT}$ time where I is an integer and the schedule loops every EATASCTM.TASOCT ns

Figure 32.25 TAS recovery during gPTP offset correction

TAS recovery time (*tas_recovery_time*) maximum value can be calculated by equation (6).

$$(6) \text{ tas_recovery_time[ns]} = \text{clk_period[ns]} \times \text{gtp_jump_time[ns]} / \text{cycle_time[ns]} + 2 \times \text{cycle_time[ns]}$$

Where:

- *clk_period* is the clock clk period

- *gptp_jump_time* is the time that has been jumped by the gPTP time
- *cycle_time* is the schedule cycle time contained in EATASCTM register.

Functions:

- For gPTP offset correction in the future, equation (6) comes from the fact that TAS switches its start time by *cycle_time* every clock during recovery. A problem resulting from that is that the recovery time increases proportionally with $tase_set_time[ns] - start_time[ns]$. To avoid this issue, TAS can switch its start time by $256^i \times cycle_time$ ($i = 0$ to 6) every clock when AVTP timer is too far in the future.

Restrictions:

- HW: Because of the gPTP offset correction, the hardware guard band cannot guarantee from the gPTP offset correction event time to the end of the first new cycle (frames can cross a gate close state)
- HW: Because of the gPTP offset correction, EAEIS1.TASGES gate errors can be set
- HW: If *gptp_jump_time* is too small (less than $2 \times cycle_time$), there is a possibility that the TAS cannot go back to its initial gate states but the recovery can still happen
- HW: For gPTP offset correction in the past, TAS cannot switch its start time by $256^i \times cycle_time$ ($i = 0$ to 6) every clock. Therefore, a gPTP offset correction too far in the past might result in a long recovery time.

32.5.1.6.2 TAS Schedule

TAS schedule defines the time of a cycle when a queue is allowed to transmit frames. A TAS schedule should be learned in the TAS RAM using TAS RAM learning functionality. This section describes the following information:

- TAS entry format
- TAS entry learning function
- TAS entry reading function
- Conversion between TAS schedule to a set of TAS entries.

1. TAS entry format

The TAS RAM is used to store TAS entries which compose the TAS schedule. A TAS entry contains the information of a gate state. All the fields in this table, if quoted, are written as TAS.{Field name}. [Table 32.6](#) describes fields contained in a TAS entry.

Table 32.6 TAS entry format

Field name	Field size (bit)	Field description
GS	1	Gate state Values: • 0: Gate state is closed • 1: Gate state is opened
GT	28	Gate time Functions: • Values of time in nanoseconds associated with the entry gate state TAS.GS

2. TAS entry learning

Learning is used to overwrite entries in the TAS RAM. [Table 32.7](#) describes register used to learn an entry in the TAS RAM. There is no learning result because learning never fails.

Table 32.7 TAS entry learn registers (1 of 2)

Register name	Field name/corresponding field in TAS RAM	Field description
EATASGL0.TASGAL	Entry address	Not present in TAS RAM, new entry is written at this address
EATASGL1.TASGSL	TAS.GS	See section 32.5.1.6. TAS (Time Aware Shaper) [802.1Qbv].

Table 32.7 TAS entry learn registers (2 of 2)

Register name	Field name/corresponding field in TAS RAM	Field description
EATASGL1.TASGTL	TAS.GT	See section 32.5.1.6. TAS (Time Aware Shaper) [802.1Qbv] .

3. TAS entry reading

Reading is used to read entries in the TAS RAM. [Table 32.8](#) describes register used to read an entry in the TAS RAM and [Table 32.9](#) describes the read results.

Table 32.8 TAS entry read registers

Register name	Field name/corresponding field in TAS RAM	Field description
EATASGR.TASGAR	Entry address	Not present in TAS RAM, entry is read from this address

Table 32.9 TAS entry read result

Register name	Field name/corresponding field in TAS RAM	Field description
EATASGRR.TASGSR	TAS.GS	See section 32.5.1.6. TAS (Time Aware Shaper) [802.1Qbv]
EATASGRR.TASGTR	TAS.GT	See section 32.5.1.6. TAS (Time Aware Shaper) [802.1Qbv]

4. Conversion between TAS schedule to a set of TAS entries

(a) Schedule example

[Figure 32.26](#) describes a basic schedule example with basic gate behavior and recommendations while designing a schedule.

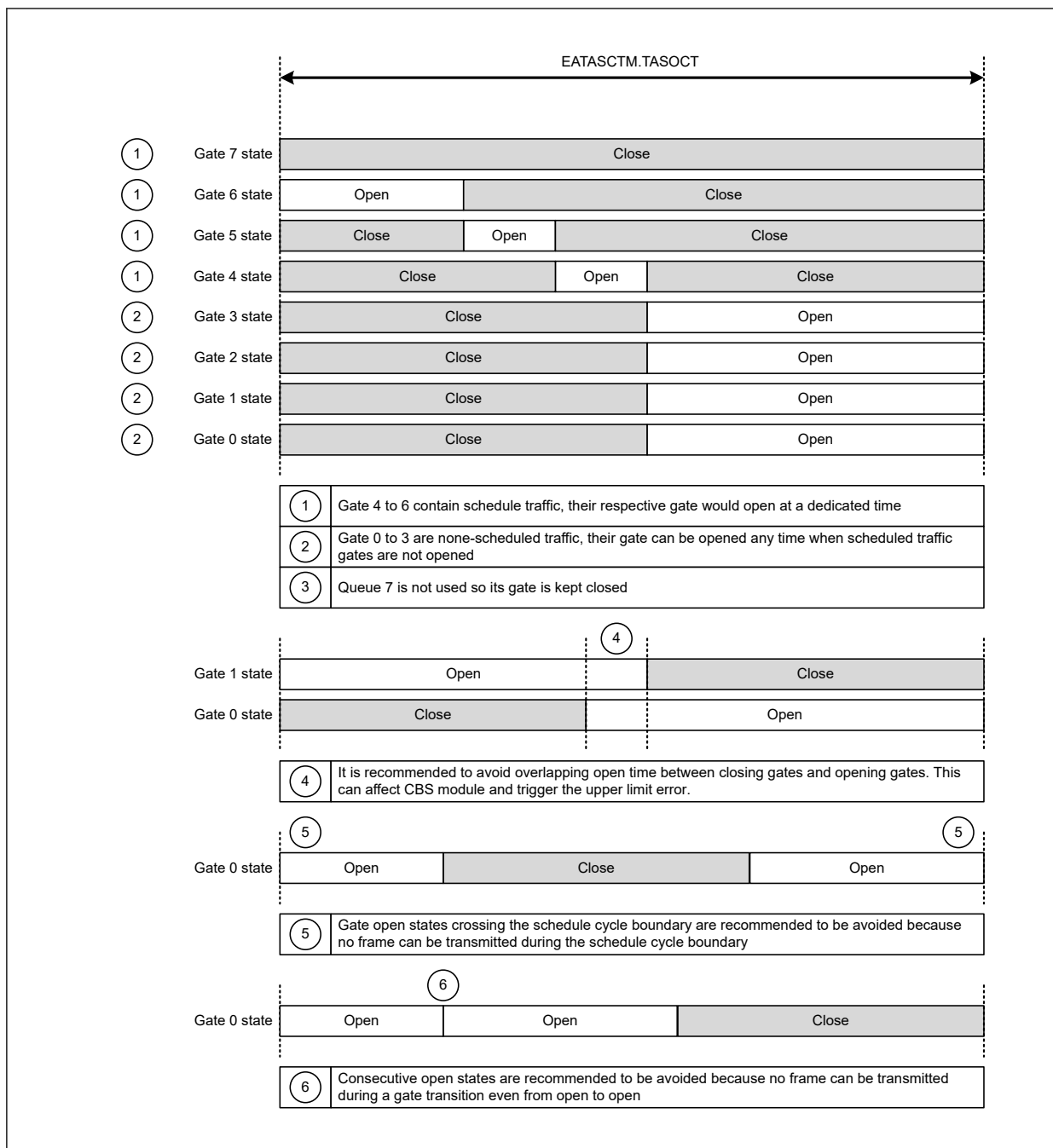


Figure 32.26

Schedule example and recommendations

Restrictions:

- SW: All p-frames queues (EATPEC.TTQ corresponding bit set to 1) should be opened simultaneously. For preemption, see [section 32.5.1.7.2. Preemption](#).

(b) Schedule setting

[Figure 32.27](#) describes how to convert a schedule into a set of TAS entries and how to set the schedule corresponding register through an example.

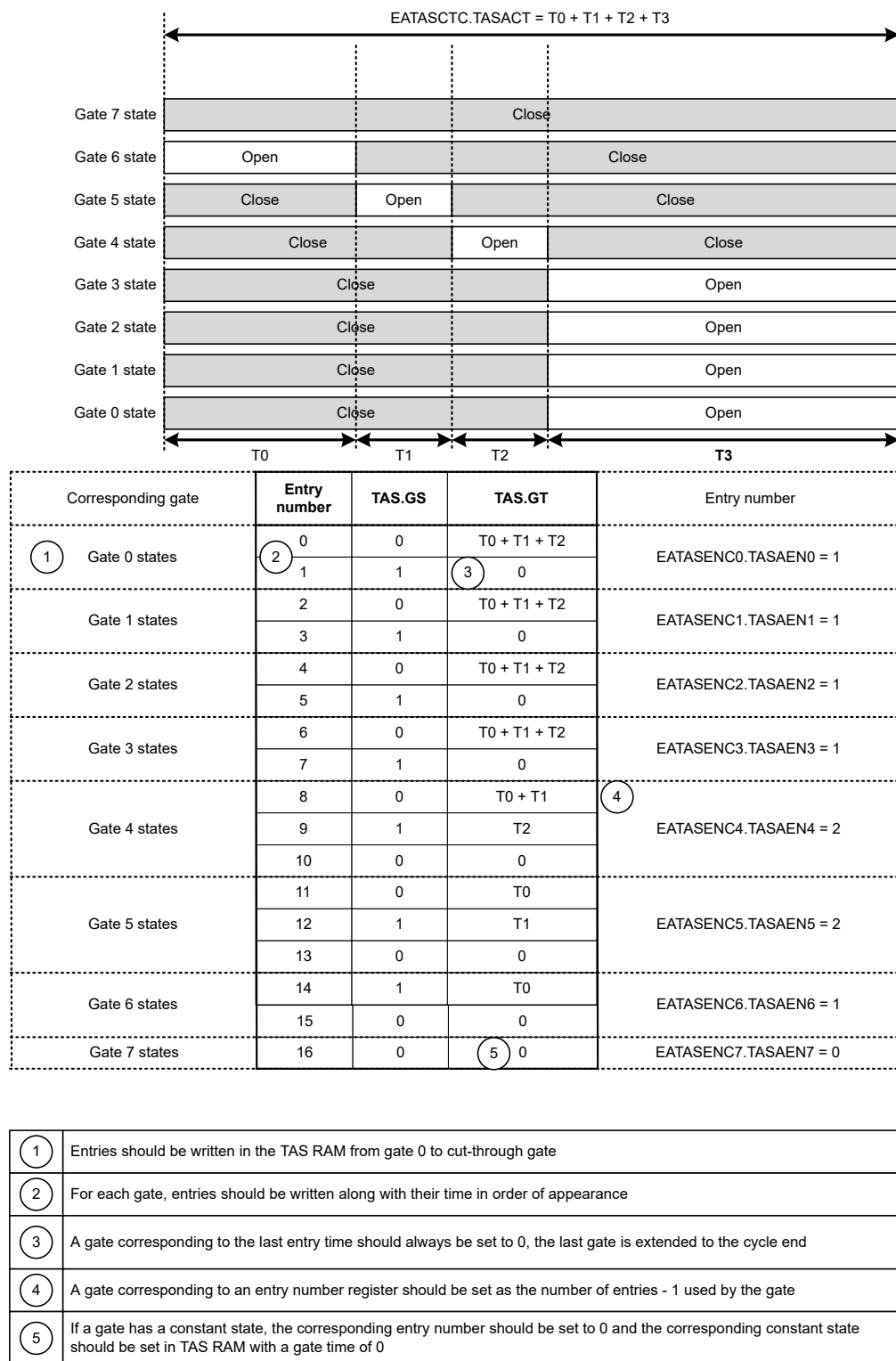


Figure 32.27

Schedule example

Restrictions:

- SW: Each gate time (T_0 , T_1 , T_2 , T_3 in Figure 32.27) should be greater or equal to $50 \text{ ns} + EATASHCC.TASJ$.

5. Gate open time calculation

Restrictions:

- SW: To avoid TAS module from clogging, gate open time should respect a minimum time value *minimumOpenTime* per queue. This time can be calculated by equation (7) for e-queues and equation (8) for p-queues (for preemption explanation, see section [section 32.5.1.7.2. Preemption](#)).

$$(7) \text{minimumOpenTime[ns]} = (\text{maximumFrameSize[byte]} \times 8 + 160) / \text{portTransmitRate[Gbps]} + \text{jitter[ns]} + \text{clk_period[ns]}$$

$$(8) \text{minimumOpenTime[ns]} = (\text{MinimumFragmentSize[byte]} \times 8 + 704) / \text{portTransmitRate[Gbps]} + \text{jitter[ns]} + \text{clk_period[ns]}$$

Where:

- *minimumOpenTime* is the smallest open time for a gate in the schedule.
- *maximumFrameSize* is the maximum frame size a queue can receive. Its value for descriptor queue q is equal to **EATMFSCq.MFS**.
- *portTransmitRate* is the ETHA link throughput [RMAC].
- *Jitter* is the internal jitter set in **EATASHCC.TASJ** register.
- *clk_period* is the clock clk period.
- *MinimumFragmentSize* is the minimum fragment size set in **EATPEC.AFS** register.

32.5.1.6.3 TAS Hardware Calibration

TAS transmission authorization is given to a frame before the frame read by TX data fetch block appends and before frame is sent to RMAC for transmission. Because of that, there is an internal minimum latency and an internal maximum jitter (Figure 32.28) that should be taken in account while setting TAS module. Latency calibration should be handled by software and jitter calibration should be set by software and handled by hardware. This section describes how to calibrate the TAS module in latency and jitter until the PHY interface [RMAC]. By adding, for example, the minimum latency and the maximum jitter until the next IP PSFP gate filtering module, it is possible to calibrate the TAS to it.

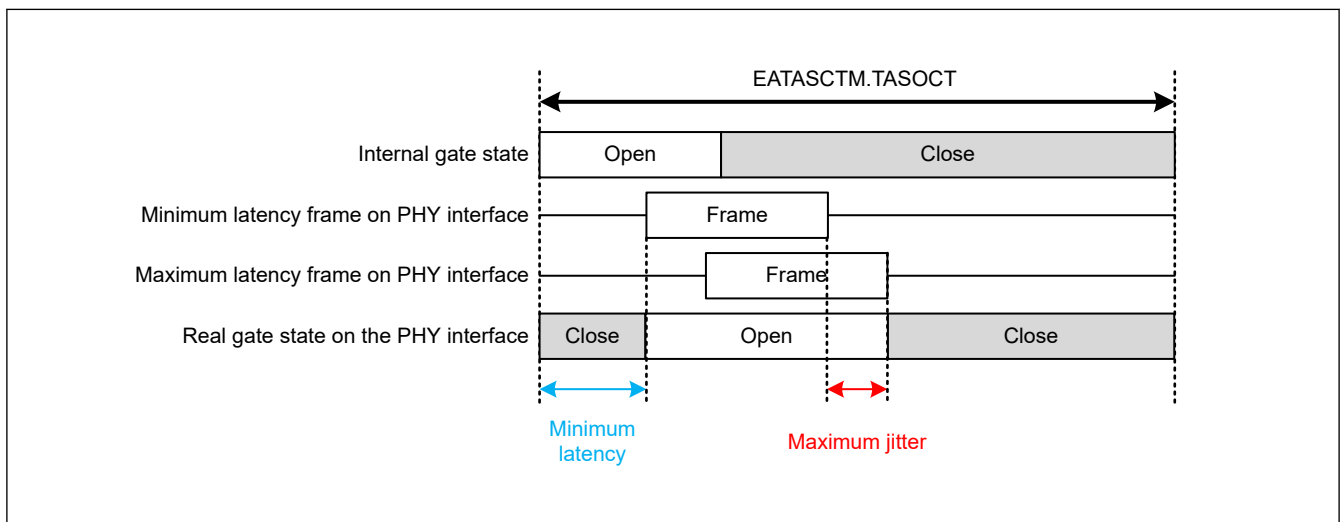
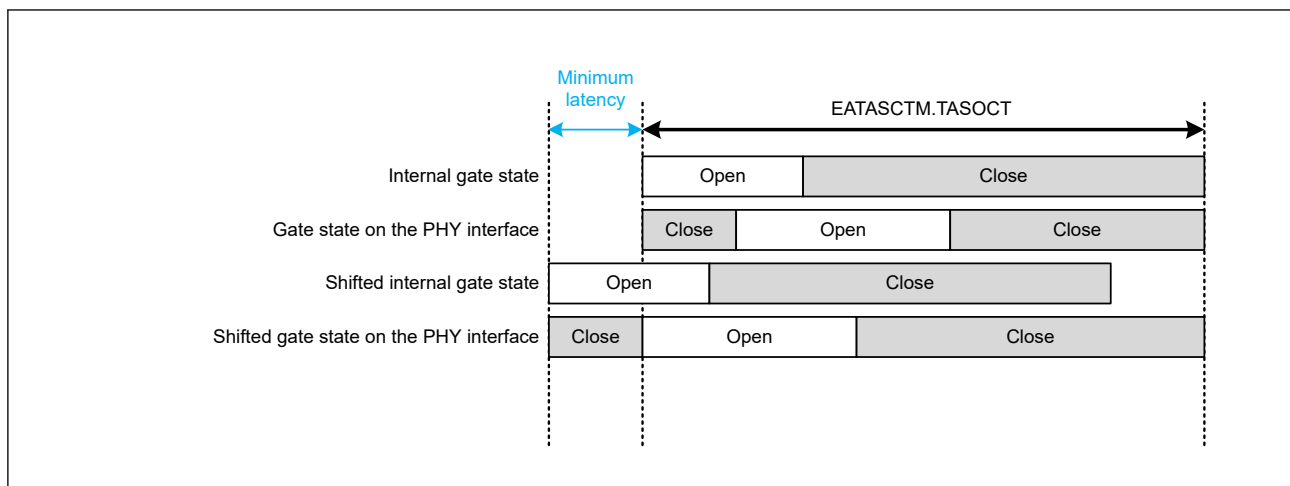


Figure 32.28 Minimum latency and maximum jitter

1. Latency calibration

Because of the internal minimum latency, with a non-calibrated schedule, a frame always starts to transmit late on the PHY interface [RMAC]. To correct this, the TAS schedule should always be in advance compared to the expected schedule of the minimum latency between the TAS transmission decision and the actual transmission on the PHY interface (Figure 32.29). Table 32.10 describes this minimum latency. To calibrate the schedule, software should always subtract the minimum latency from the cycle start time before setting it in EATASCSTC0/1 registers.

**Figure 32.29 Minimum latency calibration****Table 32.10 Minimum latency values**

PHY speed	Minimum latency when preemption disabled (EATPEC.TTQ set to All0)	Minimum latency when preemption enabled (at least one bit of EATPEC.TTQ set to 1)
10 Mbps	$8 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$	$3 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$
100 Mbps	$8 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$	$3 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$
1 Gbps	$8 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$	$3 \times clk_period[ns] + 10 \times clk_phy_tx_period[ns] + fabricLatency[ns]$

Where:

- *clk_period* is the clock clk period.
- *clk_phy_tx_period* is the clock clk_phy_tx period.
- *fabricLatency* is the latency induced by the fabric. See fabric specification document for calculation [MFAB].

2. Jitter calibration

Because of the internal maximum jitter, with a non-calibrated schedule, a frame always finishes transmitting late on the PHY interface [RMAC]. To correct this, a TAS schedule opened gate should always be closed in advance compared to the expected close time of the maximum jitter (Figure 32.30). This maximum jitter should be calculated using equation (9). To calibrate the schedule, software should set the maximum jitter in EATASHCC.TASJ register.

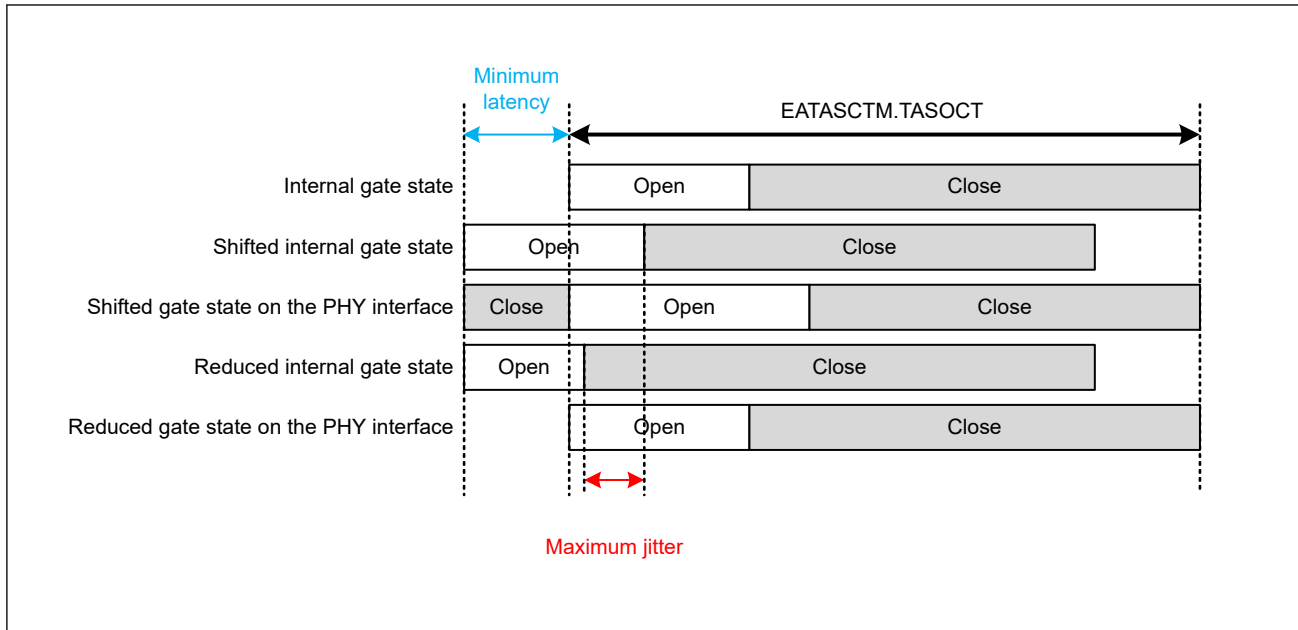


Figure 32.30 Maximum jitter calibration

$$(9) \text{ jitter[ns]} = \text{tasInternalJitter[ns]} + \text{fabricJitter[ns]} + \text{preemptionJitter[ns]} + \text{asynchronousJitter[ns]} + \text{gptpSyncJitter[ns]} + \text{clkDerivationJitter[ns]}$$

Where:

- *tasInternalJitter* is the jitter induced by TAS module and is equal to $2 \times \text{clk_period[ns]}$ where *clk_period* is the clock clk period.
- *fabricJitter* is the jitter induced by the fabric. See the Fabric specification for calculation [MFAB].
- *asynchronousJitter* is the jitter induced by the asynchronous conversion in RMAC [RMAC] and is equal to $\text{clk_period[ns]} + \text{clk_phy_tx_period[ns]}$ where *clk_period* is the clock clk period and *clk_phy_tx_periods* the clock clk_phy_tx period.
- *gptpSyncJitter* is the jitter induced by gPTP synchronization between the gPTP master clock and the gPTP slave clock.
- *clkDerivationJitter* is the jitter induced by clock derivation between the gPTP clock clk and the PHY TX clock clk_phy_tx and is equal to $\text{clkRelativeDerivation} \times \text{maxGateTime[ns]}$ where *clkRelativeDerivation* is the clock derivation and *maxGateTime* is the longest opened gate in the TAS schedule. *clkRelativeDerivation* is equal to 0 when TAS corresponding gPTP timer is the master and the gPTP clock is phase synchronous to the TX PHY clock.
- *preemptionJitter* is the jitter induced by preemption and is equal to $5 \times \text{clk_period[ns]}$ where *clk_period* is the clock clk period when preemption is enabled (at least one bit of EATPEC.TTQ register is set to 1) and 0 ns otherwise.

32.5.1.6.4 TAS Debug Interface

This interface aims at debugging the TAS gate states by giving it out of the IP.

Functions:

- `eha_race_tas_gate_state[8]` reflects the cut-through queue of the current TAS gate state.
- `eha_race_tas_gate_state[7:0]` reflects the current TAS gate states.

32.5.1.7 TX Data Fetch

TX data fetch is the same as RX data fetch in the GWCA except that it can handle preemption and also data fetching for cut-through queues. See GWCA specification for more details [GWCA] about data update for none cut-through frames.

[Table 32.11](#) describes the association between ETHA and GWCA registers/signals.

Table 32.11 TX data fetch ETHA/GWCA register correspondence

Register/signal name in GWCA [GWCA]	Register/signal name in ETHA
GWVCC.VEM	EAVCC.VEM
{GWMAC0.MAU, GWMAC1.MAL}	{MRMAC0.MAU, MRMAC1.MAL} [RMAC]
GWRGC.RCPT	Fixed to 1. RMAC adds an FCS to all frames sent without it.

32.5.1.7.1 Cut-through Frames

Cut-through frames handling by data fetched is transparent for TX data fetch. All cut-through frames has e-frames for transmission.

32.5.1.7.2 Preemption

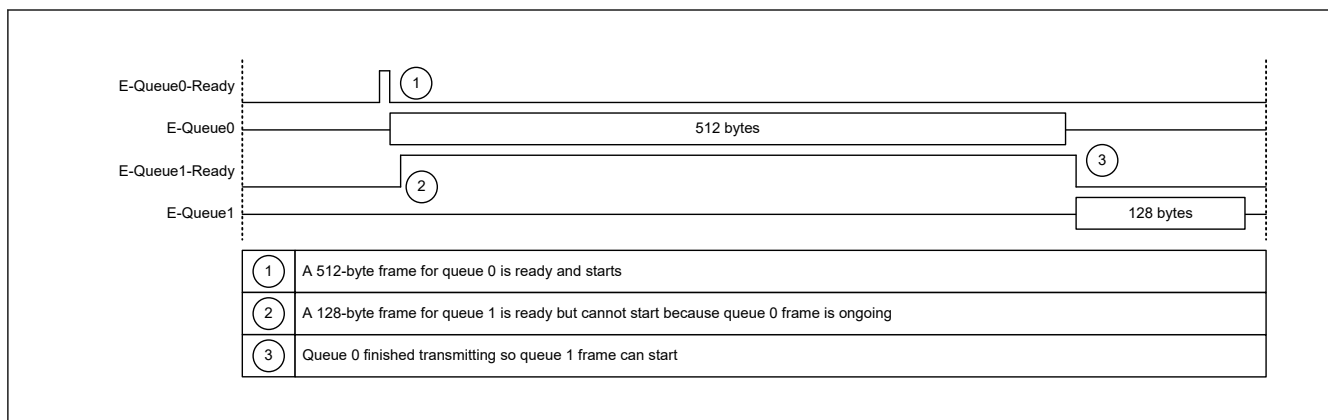
Preemption functions allow express frames (e-frames) to interrupt a preemptable frame (p-frame) to start transmitting faster (a preemptable frame can also be interrupted by TAS gate close event) using the EATPEC register. [Figure 32.31](#) describes a transmission between two e-queues, [Figure 32.32](#) describes the same transmission but with the highest priority queue being a p-queue and [Figure 32.33](#) describes how preemption occurs for a p-frame.

Functions:

- EATPEC.AFS is used to set the minimum fragment size
- EATPEC.TTQi is used to set descriptor queue q traffic to p-traffic or e-traffic.

Restrictions:

- HW: In Ethernet agent, preemption occurs on a 64-bit bus. As a result, preemption can only be performed every 16 bytes in a frame.
- HW: P-frames can only be preempted by e-frames with a higher priority. As a result, if p-frames are to be preempted by e-frames, always set low priority queues to p-frames and high priority queue to e-frames. The EAIRC register can be used to remap IPv4s coming from MFWD to p-queues or e-queues.

**Figure 32.31 E-frame transmission**

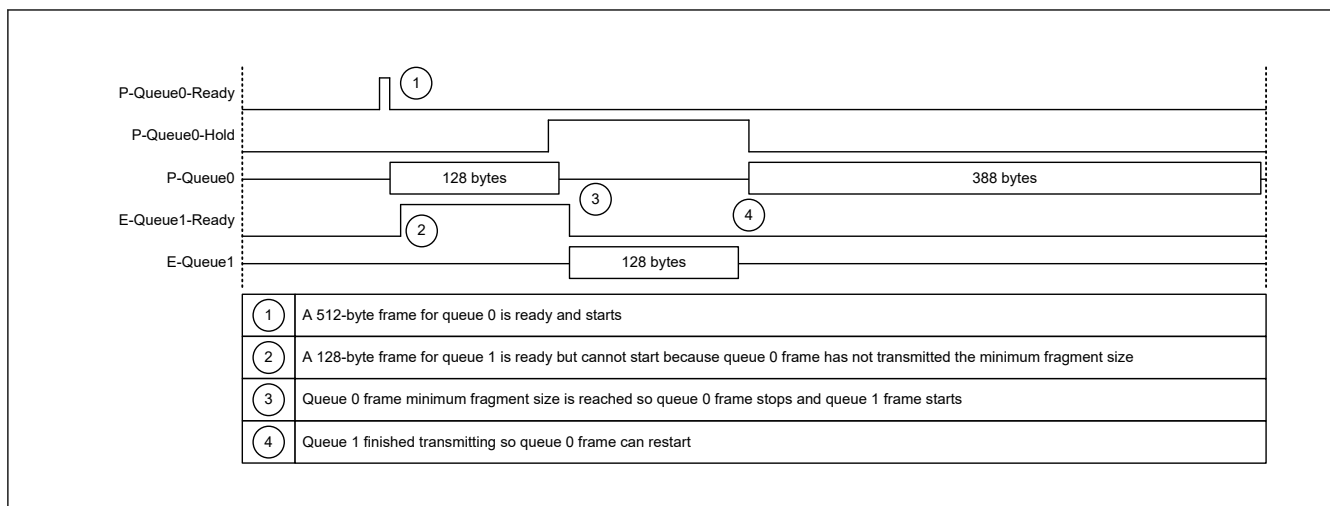


Figure 32.32 E-frame and p-frame transmission (EATPEC.AFS == 01)

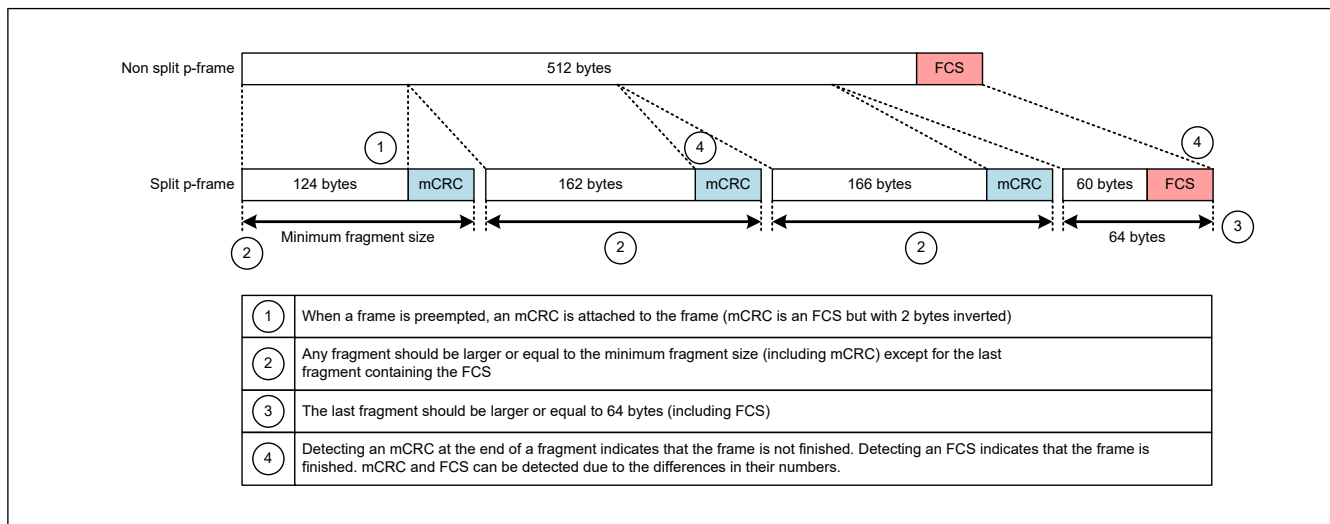


Figure 32.33 P-frame preemption (EATPEC.AFS == 01)

32.5.1.8 RMAC TX Interface Control

After TX data fetch, data is sent to the RMAC through MHD TX interface [RMAC]. Table 32.12 shows how MHD TX interface status information are provided to the RMAC.

Table 32.12 MHD TX interface status provided by ETHA to RMAC

Signal name in RMAC [RMAC]	Bus width	Value provided by ETHA
FCS in	1	Set to 1 if the corresponding transmit frame is an FCS (descriptor is received from forwarding engine with FDESCR.FI equal to 1) and if the frame has not been modified by the switch (no update due to VLAN tagging/untagging and no update requested by L3 routing [FWD])
Timestamp capture	1	Set to LDESCR.TXC
Insert egress timestamp	1	Set to LDESCR.IET
Calculate residence time	1	Set to LDESCR.CRT
Timer number	1	Set to LDESCR.TN
Timestamp unique number	10	Set to {SPN, LDESCR.TSUN} where SPN is the source port number of the corresponding frame

32.5.2 Data Reception

Ethernet agent allows data reception through the ETHA RX data path. [Figure 32.34](#) describes the RX data path.

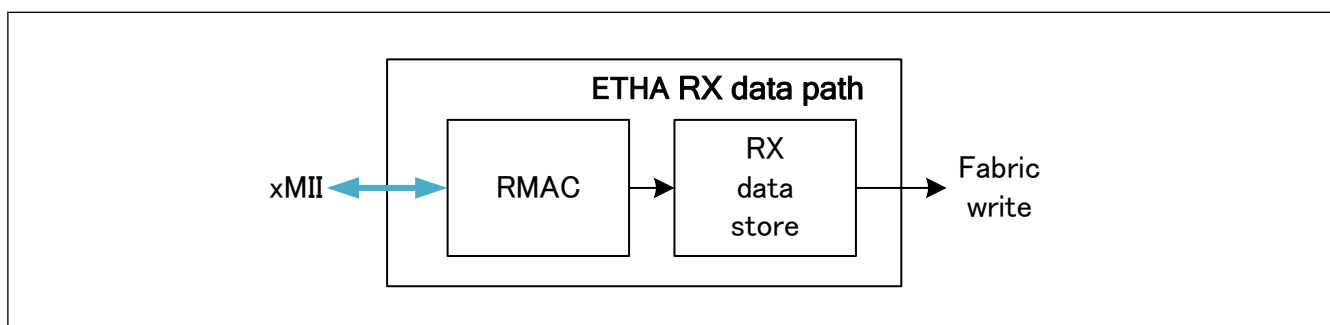


Figure 32.34 ETHA RX data path block diagram

The RX data path is separated in two blocks:

- **RMAC:** This block handles the data exchange with the Ethernet PHY. The functionalities of this block are described in the RMAC specification [RMAC].
- **RX data store:** This block extracts TAG information for frames, applies VLAN tagging and saves the frames in the local RAM. This block can also release pointers while exiting OPERATION mode. The pointer release exists for switch hardware purpose and its description is not required for switch utilization.

Restrictions:

- **SW:** The switch maximum input frame size is 60 KB. Set RMAC so it does not send frames larger than 60 KB (set MRFSC.EMXS and MRFSCP.PMXS to a value smaller or equal to 61440 [RMAC]).

32.5.2.1 RX Data Store

The function of RX data store is the same as the function of TX data store in GWCA except for the local descriptor format. See GWCA specification for more details [GWCA]. [Table 32.13](#) describes the association between ETHA and GWCA registers.

Table 32.13 Data store ETHA/GWCA register correspondence

Register/signal name in GWCA [GWCA]	Register/signal name in ETHA
GWTTFC	EARTFC
GWVCC.VIM	EAVCC.VIM
GWVTC	EAVTC
GWEIS0.USMFSES	EAEIS0.USMFSES

As local descriptor format, ETHA can only use ethernet local descriptors. Ethernet local descriptor is explained in GWCA specification and the field corresponding values for ethernet agent is described in [Table 32.14](#).

Table 32.14 Ethernet local descriptor field description (1 of 2)

Field name	Bit width	Value for ETHA
FI	1	Set to RMAC MHD Rx Interface "FCS included" field [RMAC]
FMT	1	Set to 0 for ethernet descriptors
TXC	1	Set to RMAC MHD Rx Interface "Timestamp capture at TX side" field [RMAC]
IET	1	Set to 0
CRT	1	Set to 0
TN	1	Set to RMAC MHD Rx Interface "Time domain number" field [RMAC]
TSUN	8	Set to RMAC MHD Rx Interface "Timestamp unique number" field [RMAC]

Table 32.14 Ethernet local descriptor field description (2 of 2)

Field name	Bit width	Value for ETHA
SAEF	8	See Figure 32.35 and Table 32.15
VCTRL	3	VCTRL[2] values: - Set to EAVCC.VIM - VCTRL[1:0] values 00: For No TAG, R-TAG and Unknown TAG frames 01: For C-TAG and CR-TAG frames 10: For SC-TAG and SCR-TAG frames 11: For CoS-TAG and CoSR-TAG frames
RTGI	1	Values: 0: For No TAG, CoS-TAG, C-TAG, SC-TAG and Unknown TAG frames 1: For R-TAG, CoSR-TAG, CR-TAG and SCR-TAG frames
TPL	16	Payload length of frame. Only includes data saved in the data RAM [MFAB].
TSV	1	Values: 0: Timestamp has not been received along with the frame from RMAC [RMAC] 1: Timestamp has been received along with the frame from RMAC [RMAC]
TSD	1	Set to RMAC MHD Rx Interface "Timestamp is a default timestamp" field [RMAC]
TSNS	30	Set to 0 if timestamp has not been received. Set to timestamps nanosecond part if timestamp has not been received.
TSS	32	Set to 0 if timestamp has not been received. Set to timestamps second part if timestamp has not been received.
—	--	Set to 0

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	EC1[1:0]		EC0[3:0]				—	TFE

Figure 32.35 Source agent error flag format**Table 32.15 Source agent error flag field description (1 of 2)**

Field name	Bit width	Description
TFE	1	TAG Filtering Error Set conditions: HW: When a frame does not fit the required TAG ingress format. See section 32.3.3.3. EARTFC : Reception TAG Filtering Configuration Register.
—	--	Reserved Set to 0

Table 32.15 Source agent error flag field description (2 of 2)

Field name	Bit width	Description
EC0	4	Error Code 0 [RMAC] Values: 0: No error 1: Frame received from RMAC with "PHY error" bit set 2: Frame received from RMAC with "PCH CRC error" bit set 3: Frame received from RMAC with "Nibble error" bit set 4: Frame received from RMAC with "FCS (mCRC) error" bit set 5: Frame received from RMAC with "Final fragment missing error" bit set 6: Frame received from RMAC with "C Fragment SMD Error" bit set 7: Frame received from RMAC with "C Fragment FRAG_COUNT Error" bit set 8: Frame received from RMAC with "RMAC Frame filtered" bit set 9: Frame received from RMAC with "Reception partially out of operation" bit set - Others: Reserved Restrictions: - HW: Because of the coding mechanism only one error can be flagged in this field - HW: If several errors occur for the same frame, the error with the smallest EC0 value is flagged.
EC1	2	Error Code 1 [RMAC] Values: 0: No error 1: Frame received from RMAC with "Buffer overflow error" bit set 2: Frame received from RMAC with "Undersize error" bit set 3: Frame received from RMAC with "Oversize error" bit set Restrictions: - HW: Because of the coding mechanism only one error can be flagged in this field - HW: If several errors occur for the same frame, the error with the smallest EC0 value is flagged.
—	--	Reserved Set to 0